



UNIVERSITÀ
DEGLI STUDI
FIRENZE

Scuola di Scienze Matematiche, Fisiche e Naturali
Corso di Laurea in Informatica

Tesi di Laurea

APPLICABILITÀ DI JHIPSTER PER LO SVILUPPO
GUIDATO DAI TEST, AUTOMAZIONE DELLA BUILD,
E INTEGRAZIONE CONTINUA

JHIPSTER'S APPLICABILITY FOR TEST-DRIVEN
DEVELOPMENT, BUILD AUTOMATION, AND
CONTINUOUS INTEGRATION.

GIOVANNI PAGANELLI

Relatore: *Lorenzo Bettini*

Anno Accademico 2025-2026

INDICE

Elenco delle figure	3
1 Introduzione	5
2 Stato dell'Arte	7
2.1 Test-Driven Development (TDD)	7
2.2 Behavior-Driven Development (BDD)	8
2.3 Continuous Integration	9
2.4 Generatori di applicazioni full-stack	9
2.5 Motivazione del presente lavoro	9
3 JHipster: Architettura e fondamenti teorici	11
3.1 Architettura backend: il ruolo di Spring Boot	11
3.1.1 Profili di Spring Boot generati	12
3.2 Persistenza e gestione delle relazioni	13
3.3 Architettura frontend: Angular	14
3.4 Autenticazione e autorizzazione	16
3.5 Supporto ai test	16
3.6 Containerizzazione e integrazione DevOps	16
4 Caso di studio: applicazione di gestione note	19
4.1 Setup del progetto e scelte architetturali	19
4.2 Modellazione del dominio	20
4.2.1 Relazione molti-a-molti tra Note e Tag	22
4.2.2 Gestione degli utenti e proprietà dei dati	23
4.3 Test-Driven Development nel caso di studio	23
4.3.1 Analisi dei test generati automaticamente da JHipster	24
4.3.2 Applicazione del ciclo Red-Green-Refactor su un vincolo di dominio	26
4.3.3 Testing della relazione molti-a-molti Note-Tag	29
4.4 Unit testing frontend con Jest	31
4.4.1 Esempi di test generati	32
4.5 Behavior-Driven Development con Cucumber	33
4.6 End-to-End Testing con Cypress	35
4.7 Mutation Testing con PIT	37
4.8 Performance Testing con Gatling	39
4.8.1 Struttura dei test pregenerati	40
4.8.2 Simulazione del carico	41
4.8.3 Analisi dei report di performance	42

INDICE

4.8.4	Personalizzazione degli scenari di test	44
4.8.5	Ruolo del performance testing nel contesto della tesi	44
5	Integrazione continua nel caso di studio	45
5.1	Infrastruttura CI locale basata su Docker	45
5.2	Pipeline Jenkins: struttura e strategia multi-branch	46
5.2.1	Validazione rapida su Pull Request	47
5.2.2	Suite completa su branch dev e raccolta report	47
5.2.3	Test ad alto costo computazionale	48
5.2.4	Analisi statica e Quality Gate con SonarQube	49
5.2.5	Build di rilascio e packaging in produzione	52
6	Valutazione critica del caso di studio	55
6.1	Limiti dello studio e minacce alla validità	55
6.2	Sistema di testing adottato	56
6.3	Caratterizzazione quantitativa del progetto	56
6.4	Supporto al Test-Driven Development	57
6.5	Supporto al Behavior-Driven Development e ai test E2E	58
6.6	Supporto alla Continuous Integration	58
6.7	Evoluzione delle metriche durante lo sviluppo	59
6.8	Evoluzione del framework durante lo studio	60
6.9	Osservazioni critiche emerse dal caso di studio	60
7	Conclusioni e sviluppi futuri	63
7.1	Sintesi del lavoro svolto	63
7.2	Risposta alla domanda di ricerca	64
7.3	Principali contributi della tesi	64
7.4	Sviluppi futuri	65
	Bibliografia	67

ELENCO DELLE FIGURE

Figura 1	Esempio di entità JPA generata da JHipster	13
Figura 2	Esempio di repository Spring Data JPA generato da JHipster . .	14
Figura 3	Estratto dell'albero del front-end generato per una entità	15
Figura 4	Schema concettuale del dominio e relazione multi-a-molti tra note e tag.	23
Figura 5	Estratto dell'albero dei test generati automaticamente	25
Figura 6	report di cucumber.	35
Figura 7	Panoramica del report generato da Pitest.	39
Figura 8	Panoramica del report di performance generato da Gatling al termine della simulazione.	42
Figura 10	Throughput del sistema espresso come numero di richieste al secondo durante la simulazione.	43
Figura 9	Distribuzione dei tempi di risposta durante l'esecuzione del test di carico.	43
Figura 11	Pagina principale del progetto su SonarQube con l'esito del Quality Gate.	51
Figura 12	Vista Overview di SonarQube con le principali metriche di qualità del progetto.	51
Figura 13	Sezione Issues di SonarQube con i problemi individuati dall'analisi statica del codice.	52

INTRODUZIONE

Lo sviluppo software è in continua evoluzione e negli ultimi anni è passato da modelli prevalentemente ad hoc e manuali a processi strutturati e in gran parte automatizzati. In particolare, metodologie come lo *Sviluppo guidato dai test* (TDD) e le pratiche di *Integrazione Continua* (CI) sono diventate la base per produrre software affidabile, manutenibile e scalabile [2, 8].

L'effettiva adozione di tali pratiche dipende però dagli strumenti e dai framework utilizzati in fase di progettazione e sviluppo. L'architettura iniziale del progetto, il codice generato automaticamente e le configurazioni degli strumenti di build, come *Maven*, possono infatti facilitare oppure ostacolare l'applicazione concreta di queste metodologie.

In questo contesto si inserisce *JHipster*, un framework open source per la generazione di applicazioni web full-stack, che utilizza *Spring Boot* lato backend e framework moderni come *Angular*, *React* o *Vue* lato frontend [15].

Questa tesi si propone di valutare in che misura il framework JHipster supporti:

- la progettazione, scrittura ed esecuzione di test unitari e di integrazione secondo un approccio *TDD* e *BDD*.
- la configurazione di una pipeline di *Continuous Integration* (CI) per automatizzare build, test e distribuzione.

L'analisi viene condotta attraverso la realizzazione di un'applicazione web per la gestione di note personali, caratterizzata dalla presenza di relazioni molti-a-molti tra le entità del dominio. Il progetto rappresenta un banco di prova per verificare l'integrazione tra codice generato, logica di business, strumenti di testing e automazione.

La struttura della tesi è la seguente. Nel Capitolo 2 vengono richiamati i fondamenti teorici di TDD e CI. Il Capitolo 3 è dedicato alla descrizione del framework JHipster. Nel Capitolo 4 viene presentata l'applicazione sviluppata e l'integrazione dei test nel progetto. Il Capitolo 5 descrive la configurazione della pipeline di Continuous Integration. Il Capitolo 6 riporta l'analisi comparativa dei risultati ottenuti, mentre il Capitolo 7 presenta le conclusioni del lavoro.

STATO DELL'ARTE

Nel corso degli ultimi decenni l'ingegneria del software è cambiata in modo sostanziale, sia dal punto di vista metodologico sia per quanto riguarda gli strumenti utilizzati nello sviluppo delle applicazioni. I primi modelli organizzativi, come il modello a cascata, erano basati su una successione rigida e sequenziale delle fasi di progetto. Con il tempo, tuttavia, tale impostazione ha mostrato diversi limiti, soprattutto in contesti caratterizzati da requisiti variabili e tempi di rilascio sempre più ridotti [22].

La progressiva diffusione di approcci iterativi e incrementali ha quindi spostato l'attenzione dalla sola consegna del prodotto finale alla gestione continua della qualità. Le metodologie Agile si inseriscono in questa evoluzione, proponendo cicli di sviluppo brevi, rilascio frequente di versioni intermedie e un costante confronto con il cliente [1].

In tale contesto, la qualità non è più un'attività relegata alla fase finale di testing, ma diventa un elemento trasversale all'intero ciclo di vita dell'applicazione. Di conseguenza, si è rafforzata l'adozione di pratiche orientate all'automazione dei test e dei processi di build, con l'intento di ridurre l'introduzione di difetti, migliorare la manutenibilità del codice e rendere più affidabili i rilasci [17].

2.1 TEST-DRIVEN DEVELOPMENT (TDD)

Il Test-Driven Development (TDD) nasce nell'ambito dell'Extreme Programming ed è stato formalizzato da Kent Beck [2]. L'idea centrale consiste nello scrivere i test prima del codice applicativo. Questo rovesciamento rispetto alla sequenza tradizionale obbliga lo sviluppatore a chiarire fin dall'inizio quale debba essere il comportamento osservabile del sistema, prima ancora di definirne l'implementazione [3].

Il ciclo operativo del TDD è composto da tre fasi:

- **Red:** si scrive un test relativo a una nuova funzionalità, il test fallisce, poiché la funzionalità non è ancora implementata.
- **Green:** si implementa il codice strettamente necessario per soddisfare il test.

- **Refactor**: si riorganizza il codice migliorandone la struttura interna, senza modificarne il comportamento verificato dai test.

La necessità di rendere il codice verificabile fin dall'inizio tende a favorire una struttura più modulare e una maggiore separazione delle responsabilità [18], riducendo il rischio di regressioni nelle fasi evolutive.

La letteratura evidenzia come il TDD contribuisca ad aumentare la copertura dei test e a rendere più espliciti i requisiti funzionali, poiché l'attenzione è rivolta al comportamento osservabile del sistema [7]. L'applicazione del metodo richiede tuttavia una certa disciplina e può comportare, soprattutto nelle fasi iniziali, un rallentamento percepito dello sviluppo. Inoltre, quando parte del codice è generata automaticamente tramite strumenti di scaffolding, l'approccio test-first può risultare meno immediato. In questi casi emerge una questione rilevante per questa tesi: fino a che punto la generazione automatica del codice è compatibile con uno sviluppo effettivamente guidato dai test?

2.2 BEHAVIOR-DRIVEN DEVELOPMENT (BDD)

Il Behavior-Driven Development (BDD) può essere interpretato come un'estensione del TDD con l'obiettivo di colmare il divario comunicativo tra sviluppatori e clienti non tecnici [19]. Il BDD propone la formalizzazione dei requisiti sotto forma di scenari scritti in linguaggio naturale. Una delle sintassi più diffuse è Gherkin, che utilizza lo schema:

```
Given una condizione iniziale \\  
When si verifica un'azione \\  
Then si osserva un risultato atteso
```

Tali descrizioni vengono poi collegate al codice eseguibile attraverso framework come Cucumber, che consentono di trasformare gli scenari in test automatizzati. Rispetto al TDD, il BDD opera quindi a un livello di astrazione più elevato, concentrandosi sul comportamento complessivo del sistema e sull'interazione tra i suoi componenti [21].

In applicazioni basate su architetture REST (o su microservizi), questo approccio si presta in modo particolare alla validazione di scenari end-to-end e dei vincoli di sicurezza. L'efficacia del BDD, tuttavia, è strettamente legata al contesto: in ambienti caratterizzati da un dialogo costante tra sviluppatori e clienti può facilitare l'allineamento sui requisiti, mentre in progetti individuali o di piccole dimensioni il beneficio può risultare più limitato.

2.3 CONTINUOUS INTEGRATION

Con l'aumento della complessità dei sistemi software, è emersa l'esigenza di automatizzare non solo il testing, ma l'intero processo di integrazione e rilascio. La Continuous Integration (CI) consiste nell'integrare frequentemente le modifiche in un repository condiviso, attivando automaticamente procedure di compilazione ed esecuzione dei test [8, 3]. In questo modo è possibile intercettare rapidamente eventuali errori e ridurre l'accumulo di conflitti tra contributi differenti.

In una pipeline CI tipica, le fasi di compilazione, testing, analisi statica e generazione degli artefatti sono completamente automatizzate e ripetibili. L'adozione di tecnologie di containerizzazione, come Docker, contribuisce ulteriormente a garantire la coerenza tra ambiente di sviluppo, testing e produzione.

L'automazione sistematica dei processi rappresenta uno degli elementi centrali del paradigma DevOps, che mira a ridurre la separazione tradizionale tra sviluppo e gestione operativa [13]. In questo contesto, la presenza di una suite di test solida e affidabile non è un elemento opzionale, ma una condizione necessaria: senza test automatici adeguati, l'integrazione continua perderebbe gran parte del suo valore operativo.

2.4 GENERATORI DI APPLICAZIONI FULL-STACK

Negli ultimi anni si sono diffusi strumenti in grado di generare automaticamente applicazioni complete a partire da un modello di dominio. L'obiettivo principale di questi generatori è ridurre il tempo dedicato alle attività preliminari di configurazione e fornire una struttura coerente e standard rispetto a pratiche consolidate.

Questi strumenti non si limitano alla produzione del codice applicativo, ma includono spesso configurazioni per la gestione della build, l'organizzazione dei livelli applicativi e, in alcuni casi, il supporto a meccanismi di testing e automazione.

Un esempio di questi strumenti è JHipster, generatori open source di cui parlerò più approfonditamente nel prossimo capitolo [15].

2.5 MOTIVAZIONE DEL PRESENTE LAVORO

I generatori di applicazioni vengono generalmente valutati in termini di produttività, riduzione del codice boilerplate e aderenza a standard consolidati.

Il presente lavoro si propone di valutare in modo sistematico la compatibilità tra generazione automatica del codice e metodologie di sviluppo orientate ai test e all'integrazione continua, specialmente nel caso di JHipster.

JHIPSTER: ARCHITETTURA E FONDAMENTI TEORICI

JHipster è una piattaforma open source progettata per la generazione di applicazioni web moderne full-stack. Il suo obiettivo è fornire una base pronta all'uso che integri tecnologie consolidate lato backend e frontend. Nel tempo, il progetto si è evoluto fino a diventare un ecosistema strutturato, mantenuto da una vasta comunità di sviluppatori [15].

Attraverso una interfaccia a riga di comando, si risponde a una serie di domande che definiscono le caratteristiche principali dell'applicazione: tipologia di architettura (monolite o microservizi), sistema di autenticazione, database, strumenti di build e framework frontend, e alcuni strumenti aggiuntivi. Si può in seguito, mediante la definizione di un modello di dominio tramite JDL (JHipster Domain Language), modellare entità, relazioni e vincoli in modo strutturato.

JHipster genera automaticamente un'applicazione completa, comprensiva di backend REST, frontend SPA (Single Page Application), sistema di autenticazione, configurazione del database, test unitari e di integrazione, e strumenti di containerizzazione.

Le informazioni relative all'installazione e l'utilizzo di JHipster possono essere trovate sul loro sito ufficiale www.jhipster.tech. I progetti generati con JHipster supportano nativamente gli IDE più diffusi, come Eclipse usato in questa tesi, e la loro documentazione spiega come utilizzare tutti i componenti integrati dal framework.

3.1 ARCHITETTURA BACKEND: IL RUOLO DI SPRING BOOT

Il backend generato da JHipster utilizza Spring Boot, un framework dell'ecosistema Spring progettato per semplificare la configurazione e l'avvio di applicazioni Java. Spring Boot riduce la complessità di Spring tramite il principio di *convention over configuration*, fornendo configurazioni predefinite che consentono di avviare rapidamente un'applicazione funzionante [24].

L'architettura backend risultante è tipicamente stratificata in più livelli. Il *domain layer* contiene le entità JPA che rappresentano il modello dati. Il *repository layer*, basato su Spring Data JPA, fornisce l'accesso ai dati mediante interfacce che estendono

`JpaRepository`. Il *service layer* incapsula la logica di business, mentre il *web layer* espone le API REST.

Questa separazione rende il codice più facilmente testabile, poiché consente di isolare la logica applicativa dal livello di persistenza e dal livello di presentazione. Dal punto di vista del TDD, la presenza di un service layer ben definito rappresenta un punto di ingresso naturale per la scrittura di test unitari.

3.1.1 Profili di Spring Boot generati

JHipster genera automaticamente diversi profili per Spring Boot. I profili permettono di definire configurazioni differenti dell'applicazione a seconda dell'ambiente di esecuzione.

In generale, i profili generati automaticamente da JHipster sono già configurati in modo adeguato per la maggior parte dei casi d'uso. Tuttavia, durante lo sviluppo di un'applicazione reale è spesso necessario personalizzare alcune configurazioni. Le modifiche più comuni riguardano principalmente:

- la configurazione del database, ad esempio sostituendo il database embedded utilizzato in sviluppo con un database relazionale esterno;
- parametri di logging differenti tra ambiente di sviluppo e produzione;
- configurazioni di sicurezza e gestione delle sessioni;
- parametri di connessione a servizi esterni, come sistemi di messaggistica o storage.

JHipster genera un file di configurazione generale, `application.yml`, che contiene le proprietà comuni dell'applicazione, e due profili principali: `dev` e `prod`.

- **application.yml**: file di configurazione base, comune a tutti gli ambienti. Contiene le proprietà generali dell'applicazione e viene integrato o sovrascritto dai profili specifici.
- **dev**: utilizzato durante lo sviluppo locale. Include configurazioni per logging più verboso, strumenti di sviluppo e un database locale.
- **prod**: destinato all'ambiente di produzione. Include configurazioni con maggiore attenzione a prestazioni, sicurezza e integrazione con servizi esterni.

Nel caso del profilo `dev`, JHipster attiva inoltre automaticamente il profilo accessorio `api-docs`, utile per l'esposizione della documentazione delle API durante lo sviluppo.

3.2 PERSISTENZA E GESTIONE DELLE RELAZIONI

La persistenza dei dati è affidata a Spring Data JPA e Hibernate. Le entità vengono annotate secondo lo standard JPA e mappate su tabelle relazionali [12].

Uno degli aspetti più rilevanti è la gestione automatica delle relazioni tra entità, incluse relazioni molti-a-molti. Tramite JDL (o CLI interattiva) è possibile definire in modo dichiarativo attributi, vincoli e relazioni (OneToOne, OneToMany, ManyToOne, ManyToMany), DTO, service class, paginazione, filtraggio. Il generatore produce automaticamente le annotazioni JPA appropriate e, nel caso delle relazioni molti-a-molti, crea la tabella di join necessaria.

La Figura 1 mostra un esempio di entità generata automaticamente dal framework.

```
/**
 * JHipster JDL model for myApp
 */
@Entity
@Table(name = "tag")
@Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
@SuppressWarnings("common-java:DuplicatedBlocks")
public class Tag implements Serializable {

    private static final long serialVersionUID = 1L;

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id")
    private Long id;

    @NotNull
    @Size(min = 1, max = 255)
    @Column(name = "tag_name", length = 255, nullable = false)
    private String tagName;

    @Size(max = 20)
    @Column(name = "color", length = 20)
    private String color;

    @ManyToOne(optional = false)
    @NotNull
    private User owner;

    @ManyToMany(fetch = FetchType.LAZY, mappedBy = "tags")
    @Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
    @JsonIgnoreProperties(value = { "attachments", "owner", "tags" }, allowSetters = true)
    private Set<Note> notes = new HashSet<>();
}
```

Figura 1: Esempio di entità JPA generata da JHipster

Le annotazioni come @Entity, @Table, @Id e @GeneratedValue permettono di definire la mappatura tra oggetti Java e tabelle del database. Questo consente a Spring Data JPA di generare automaticamente molte delle query necessarie per l'accesso ai dati. Le interfacce repository estendono infatti JpaRepository, che fornisce automaticamente implementazioni per le operazioni CRUD più comuni.

È inoltre possibile definire query semplicemente dichiarando metodi con nomi specifici all'interno dell'interfaccia repository. Spring Data JPA analizza la firma del metodo e genera automaticamente la query SQL corrispondente.

La Figura 2 mostra un esempio di repository per l'entità Tag.

```
/**
 * Spring Data JPA repository for the Tag entity.
 */
@Repository
public interface TagRepository extends JpaRepository<Tag, Long> {
    @Query("select tag from Tag tag where tag.owner.login = ?#{authentication.name}")
    List<Tag> findByOwnerIsCurrentUser();

    List<Tag> findDistinctAllByNotesId(@Param("noteId") Long noteId);

    Page<Tag> findAllByOwnerLogin(String login, Pageable pageable);

    Page<Tag> findDistinctAllByOwnerIdAndTagNameNotInAndTagNameStartingWithOrderByTagNameAsc(
        @Param("ownerId") Long ownerId,
        @Param("notetags") Collection<String> notetagname,
        @Param("initial") String initial,
        Pageable pageable
    );

    Page<Tag> findDistinctAllByOwnerIdAndTagNameStartingWithOrderByTagNameAsc(
        @Param("ownerId") Long ownerId,
        @Param("initial") String initial,
        Pageable pageable
    );

    @EntityGraph(attributePaths = "notes")
    Optional<Tag> findOneWithNotesByIdAndOwnerLogin(Long id, String login);

    @EntityGraph(attributePaths = "notes")
    Optional<Tag> findOneWithNotesById(Long id);

    Optional<Tag> findOneByIdAndOwnerLogin(@Param("id") Long id, @Param("login") String login);

    Optional<Tag> findOneByTagNameAndOwnerLogin(@Param("tagName") String tagName, @Param("login") String login);

    long countByIdInAndOwnerLogin(Collection<Long> ids, String login);

    boolean existsByTagNameAndOwnerLogin(@Param("tagName") String tagName, @Param("login") String login);
}
```

Figura 2: Esempio di repository Spring Data JPA generato da JHipster

3.3 ARCHITETTURA FRONTEND: ANGULAR

JHipster supporta diversi framework frontend, tra cui: Angular, React, Vue. In questo progetto viene utilizzato Angular. L'applicazione generata da JHipster è una Single Page Application (SPA) che comunica con il backend esclusivamente tramite API REST.

Angular è un framework strutturato basato su TypeScript, con Angular è possibile realizzare componenti, servizi e direttive che cooperano tra loro per costruire applicazioni client complesse. La comunicazione con il backend avviene tramite il modulo HttpClient, che gestisce richieste asincrone e serializzazione JSON [10].

Il frontend generato include:

- componenti CRUD per ciascuna entità;
- servizi dedicati alla comunicazione con le API REST;

- gestione del routing e delle guardie di autenticazione;
- integrazione con il sistema di sicurezza JWT.

```

note/
├── note.model.ts
├── note.routes.ts
├── note.test-samples.ts
├── delete/
│   ├── note-delete-dialog.component.html
│   ├── note-delete-dialog.component.spec.ts
│   └── note-delete-dialog.component.ts
├── detail/
│   ├── note-detail.component.html
│   ├── note-detail.component.spec.ts
│   └── note-detail.component.ts
├── list/
│   ├── note.component.html
│   ├── note.component.spec.ts
│   └── note.component.ts
├── route/
│   ├── note-routing-resolve.service.spec.ts
│   └── note-routing-resolve.service.ts
├── service/
│   ├── note.service.spec.ts
│   └── note.service.ts
└── update/
    ├── note-form.service.spec.ts
    ├── note-form.service.ts
    ├── note-update.component.html
    ├── note-update.component.spec.ts
    └── note-update.component.ts

```

Figura 3: Estratto dell'albero del front-end generato per una entità

La struttura evidenzia come JHipster organizzi automaticamente i componenti dell'interfaccia separando le responsabilità: la cartella `list` contiene i componenti per la visualizzazione delle entità, `detail` per la visualizzazione dei dettagli, `update` include i componenti e i servizi necessari alla creazione e modifica dei dati, mentre le cartelle `service` e `route` raccolgono rispettivamente la logica di comunicazione con il backend e quella di gestione del routing.

Dal punto di vista del testing, Angular supporta test unitari mediante strumenti come Jest, mentre per i test end-to-end JHipster integra Cypress.

3.4 AUTENTICAZIONE E AUTORIZZAZIONE

La sicurezza dell'applicazione è gestita tramite Spring Security [25]. JHipster consente di scegliere tra vari metodi di autenticazione, i più semplici e supportati out of the box sono autenticazione basata sulla sessione e tramite JSON Web Tokens (JWT) [16].

La documentazione ufficiale di JHipster illustra inoltre come implementare e integrare gli altri metodi di autenticazione più complessi, e come renderli tutti più sicuri per l'ambiente di produzione.

Il sistema inoltre implementa ruoli predefiniti come `ROLE_USER` e `ROLE_ADMIN` per gli utenti. Le restrizioni di accesso vengono applicate sia a livello di endpoint REST sia a livello di frontend tramite guardie di routing.

3.5 SUPPORTO AI TEST

Uno degli elementi distintivi di JHipster è la generazione automatica di test backend e frontend.

Per il backend vengono prodotti test basati su JUnit, Spring Test, MockMvc e Mockito. Ogni entità include test di integrazione per le operazioni CRUD, verifiche sulle validazioni e controlli sui codici di risposta HTTP.

Per il frontend Angular vengono generati test unitari per i componenti e servizi, e configurazioni per test end-to-end con Cypress.

Cypress è uno strumento moderno per test E2E che esegue il browser in modo controllato e consente di simulare interazioni. I test Cypress verificano il comportamento complessivo dell'applicazione, dall'interfaccia fino alla persistenza dei dati [5].

Inoltre, al momento della creazione dell'applicazione, è possibile specificare l'utilizzo di test di performance tramite Gatling e test BDD tramite Cucumber.

Nel prossimo capitolo verranno analizzati più nel dettaglio i test generati automaticamente da JHipster e le modalità con cui è possibile estenderli per supportare un processo di sviluppo basato sul paradigma Test Driven Development (TDD).

3.6 CONTAINERIZZAZIONE E INTEGRAZIONE DEVOPS

JHipster integra nativamente configurazioni Docker e file docker-compose per l'avvio coordinato di backend, frontend e database, con la possibilità di creazione tramite

CLI. È inoltre possibile generare configurazioni per Kubernetes e file di pipeline per strumenti di integrazione GitHub Actions, GitLab CI o Jenkins.

La documentazione ufficiale di JHipster fornisce una spiegazione dettagliata su come realizzare il proprio Docker compose e su come utilizzare i docker compose pregenerati per gli strumenti aggiuntivi come Sonar.

CASO DI STUDIO: APPLICAZIONE DI GESTIONE NOTE

Questo capitolo descrive il caso di studio utilizzato per valutare concretamente l'applicabilità di metodologie TDD, BDD e pratiche di integrazione continua in un progetto generato con JHipster. L'obiettivo non è presentare un prodotto completo in senso commerciale, bensì un dominio sufficientemente realistico e con complessità adeguata per mettere in evidenza vantaggi e limiti nell'utilizzare JHipster.

L'applicazione è ispirata, a livello concettuale, a servizi di gestione delle note come Google Keep. L'app dovrà consentire a tutti gli utenti di creare, modificare ed eliminare note, organizzandole tramite etichette (tag) e associando eventuali allegati.

4.1 SETUP DEL PROGETTO E SCELTE ARCHITETTURALI

Il progetto è stato generato tramite il CLI JHipster v8.11.0 come applicazione monolitica, scelta motivata dal fatto che consente di concentrare l'analisi sulle pratiche di testing e automazione, evitando la complessità aggiuntiva tipica di un'architettura a microservizi. Di seguito sono riportate le principali opzioni rese disponibili dal CLI durante la fase di generazione del progetto:

- Nome dell'applicazione: **NoticeMe**;
- Architettura: **Monolitica**, Gateway, Microservizi;
- Strumento di build per il backend: **Maven**, Gradle;
- Nome del package Java di base: **com.giovannip.noticeme**;
- Supporto a Spring WebFlux: Yes/**No**;
- Tipo di autenticazione: Session-based, OAuth 2.0 / OIDC, **JWT**;
- Framework di test aggiuntivi per il backend: **Cucumber**, **Gatling**;
- Tipo di database: Cassandra, MongoDB, Neo4j, Couchbase, **SQL**;

- Database di sviluppo: H2 in-memory, **H2 su disco**, MySQL;
- Database di produzione: PostgreSQL, MariaDB, **MySQL**, altri;
- Cache provider: Hazelcast, Caffeine, Infinispan, **Ehcache**;
- Second-level cache di Hibernate: No, **Yes**;
- Tecnologie aggiuntive: Elasticsearch, WebSockets, Kafka, Pulsar, API first development con OpenAPI;
- Framework client: React, Vue, **Angular**, nessun frontend;
- Framework di test aggiuntivi per il frontend: **Cypress**;
- Interfaccia di amministrazione: No, **Yes**;
- Tema grafico client: Brite, Flatly, Cerulean, ecc., **Default**;
- Supporto alla traduzione: No, **Yes**;
- Lingua nativa: **it**;
- Lingue aggiuntive abilitate: **en**;
- Test Cypress: No, **Yes**;
- Audit Cypress: No, **Yes**.

4.2 MODELLAZIONE DEL DOMINIO

Per generare il dominio dell'applicazione è possibile ricorrere al CLI di JHipster tramite il comando `jhipster entity <entityName> -[options]`, generando le entità una per una, oppure utilizzare l'editor online di JHipster Domain Language Studio <https://start.jhipster.tech/jdl-studio/> che consente di definire direttamente entità, attributi, vincoli di validazione e relazioni tra esse. Per ciascuna entità possono inoltre essere specificate le seguenti opzioni:

- **DTO**: generazione dei Data Transfer Object tramite MapStruct;
- **Service layer**: generazione automatica delle classi di servizio per incapsulare la logica applicativa;
- **Pagination**: configurazione della modalità di paginazione nel frontend;
- **Filtering**: configurazione dei filtri disponibili sulle entità.

Il dominio dell'applicazione *NoticeMe* è stato realizzato tramite JDL Studio ed è composto dalle entità principali Note, Tag e Attachment, oltre all'entità User fornita automaticamente dal framework JHipster per la gestione dell'autenticazione e degli utenti.

Listing 1: Modello di dominio definito tramite JHipster Domain Language

```
/**
 * JHipster JDL model for NoticeMe
 */

entity Tag {
  tagName String required minlength(1) maxlength(255)
  color String maxlength(20)
}

entity Note {
  title String maxlength(255)
  content TextBlob
  lastUpdateDate Instant required
  alarmDate Instant
  status NoteStatus required
}

entity Attachment {
  fileName String required
  data AnyBlob required
  dataContentType String required
  size Long
}

enum NoteStatus{
  NORMAL,
  PINNED,
  ARCHIVED,
  DELETED
}

relationship OneToMany {
  Note{attachment} to Attachment{note required}
}

relationship ManyToOne {
```

```
Note{owner required} to User with builtInEntity,  
Tag{owner required} to User with builtInEntity,  
}  
  
relationship ManyToMany {  
    Note{tag(tagName)} to Tag{notes}  
}  
  
use infinite-scroll for Note, Tag, Attachment  
  
dto * with mapstruct  
  
service * with serviceClass
```

4.2.1 Relazione molti-a-molti tra Note e Tag

La relazione molti-a-molti tra Note e Tag permette di modellare il seguente comportamento: una nota può essere classificata con più etichette e una stessa etichetta può essere riutilizzata su più note. Dal punto di vista della persistenza, la relazione è implementata tramite una tabella di join generata automaticamente dal framework.

A livello JPA, ciò si traduce nell'uso di `@ManyToMany` e della configurazione della join table. In assenza di requisiti particolari, lo sviluppatore non deve intervenire manualmente su tale configurazione, che risulta già funzionante *out of the box*.

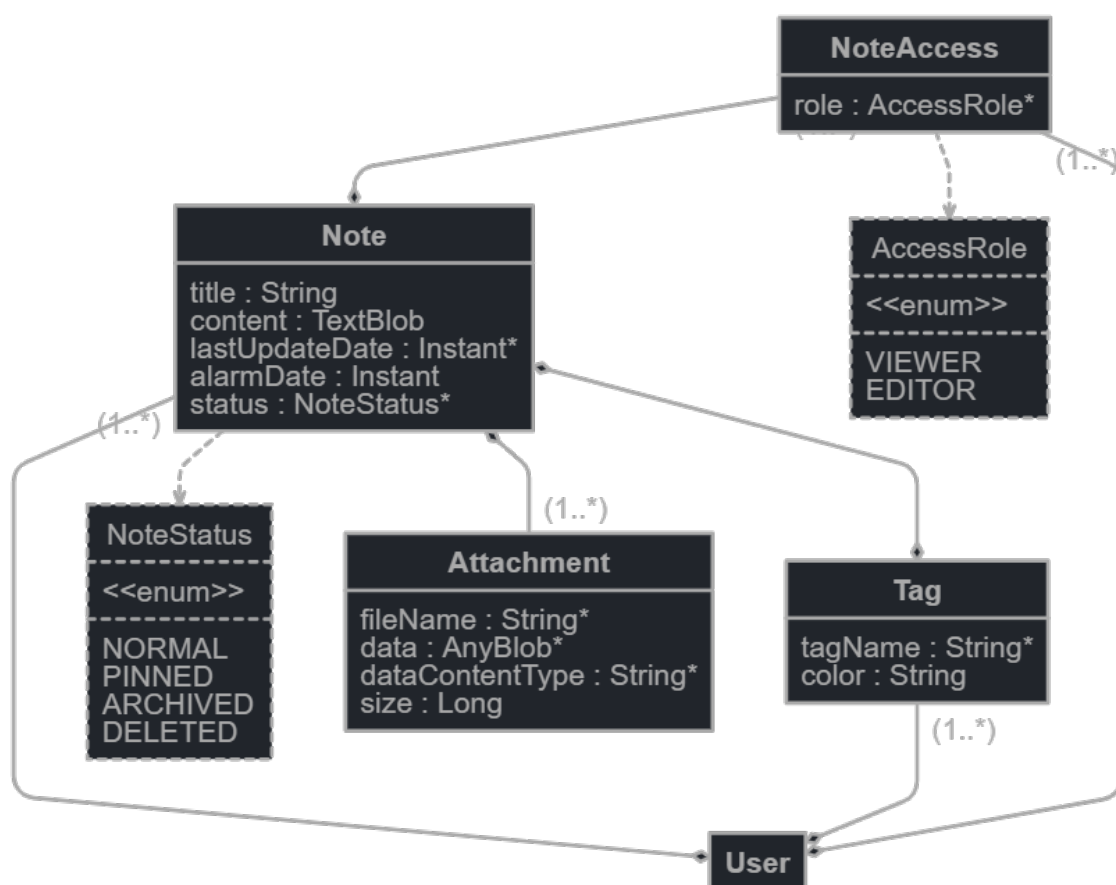


Figura 4: Schema concettuale del dominio e relazione multi-a-molti tra note e tag.

4.2.2 Gestione degli utenti e proprietà dei dati

L'applicazione sfrutta il sistema di autenticazione integrato in JHipster per associare note e tag ai rispettivi utenti proprietari. Questo consente di garantire l'isolamento dei dati tra utenti differenti e di introdurre controlli di accesso negli endpoint REST, evitando che un utente possa leggere o modificare risorse non di sua proprietà.

4.3 TEST-DRIVEN DEVELOPMENT NEL CASO DI STUDIO

Questa sezione analizza l'applicazione del Test-Driven Development (TDD) all'interno del caso di studio sviluppato con JHipster. L'analisi viene condotta attraverso l'introduzione di test aggiuntivi relativi a vincoli di dominio non coperti dal semplice scaffolding CRUD.

Il TDD prevede che la scrittura del test preceda l'implementazione della funzionalità [2, 3]. In un progetto generato con JHipster, tuttavia, una parte significativa del codice applicativo e una suite iniziale di test vengono prodotti automaticamente. Ciò non rende impossibile l'adozione del TDD fin da subito: il TDD viene applicato in modo *incrementale* sulle funzionalità aggiuntive o sui vincoli che emergono durante l'evoluzione del progetto.

4.3.1 *Analisi dei test generati automaticamente da JHipster*

JHipster genera automaticamente una suite di test backend che copre principalmente le funzionalità CRUD e la correttezza degli endpoint REST. In particolare, per ciascuna entità vengono tipicamente prodotti test di integrazione basati su Spring Boot Test e MockMvc, che verificano l'accesso alle API, la serializzazione JSON, i codici di risposta HTTP e alcune validazioni di base.

Questi test risultano utili perché forniscono una base iniziale che intercetta rapidamente errori di configurazione, verifica che l'applicazione sia avviabile e funzionante, costituisce un primo strato di regressione automatica.

Tuttavia, tali test non derivano direttamente da requisiti di dominio espliciti, ma verificano prevalentemente la correttezza strutturale dell'applicazione (ad esempio: "l'endpoint di creazione restituisce 201", "il campo obbligatorio non può essere nullo"). Inoltre, non coprono regole di business specifiche, che devono essere introdotte e testate manualmente.

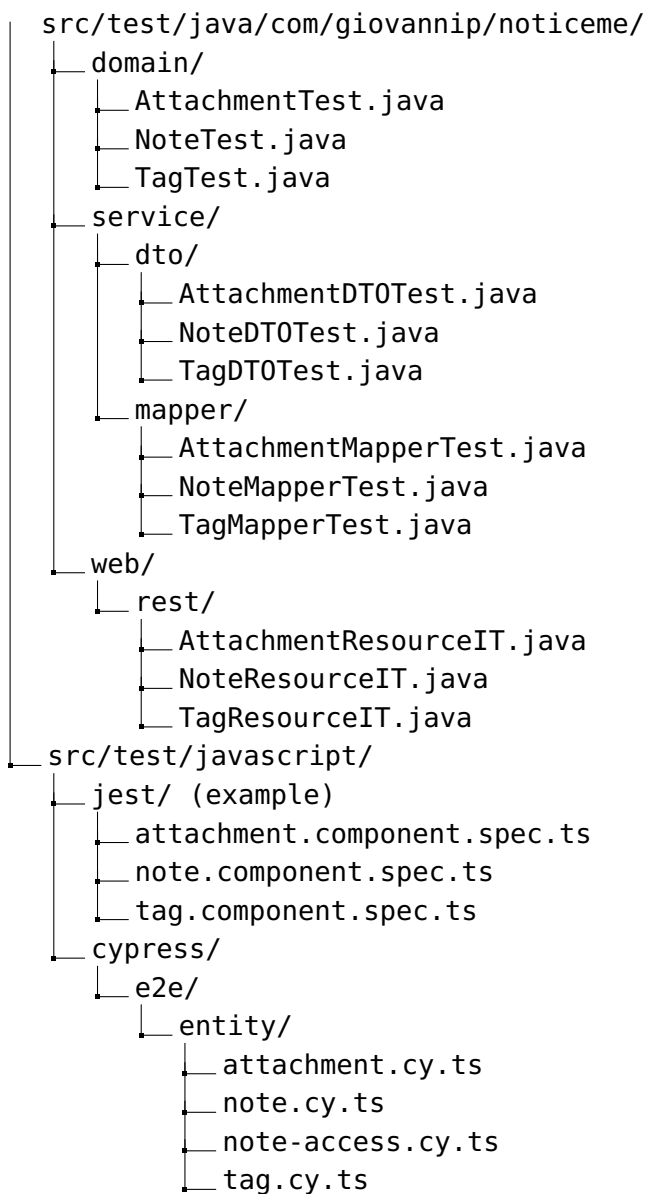


Figura 5: Estratto dell'albero dei test generati automaticamente

JHipster genera automaticamente una suite di test che copre sia il backend che il frontend dell'applicazione. In particolare sono presenti test di integrazione per le API REST, test unitari per dominio, DTO e mapper, test unitari frontend tramite Jest e test end-to-end realizzati con Cypress.

Test di integrazione con Spring Boot

Oltre ai test unitari, l'applicazione utilizza test di integrazione per verificare il comportamento del sistema con il contesto applicativo completo, includendo controller REST, service layer, repository e configurazione di sicurezza.

Nel progetto sviluppato con JHipster, i test di integrazione sono organizzati principalmente nelle classi `*ResourceIT`, generate automaticamente per verificare gli endpoint REST.

4.3.2 Applicazione del ciclo Red–Green–Refactor su un vincolo di dominio

Per applicare il TDD, consideriamo un vincolo di business non garantito automaticamente dallo scaffolding delle entità: *un utente può associare a una nota esclusivamente tag di sua proprietà*. È quindi necessario introdurre una logica applicativa nel service layer (o in uno strato equivalente) che verifichi la proprietà dei tag prima di effettuare l'associazione.

Fase Red: definizione del comportamento atteso

Nella fase iniziale è stato scritto un test di integrazione che formalizza il comportamento desiderato: dato un utente autenticato, proprietario della nota, l'aggiornamento deve fallire se tra i tag associati compare almeno un tag appartenente a un altro utente.

Listing 1: Test TDD: rifiuto dell'associazione di un tag appartenente a un altro utente

```
@Test
@Transactional
void putNoteShouldRejectTagOwnedByAnotherUser() throws Exception {
    User owner = getOrCreateUser(em, "user");
    User other = getOrCreateUser(em, "other");

    insertedNote = noteRepository.saveAndFlush(
        new Note()
            .title(DEFAULT_TITLE)
            .content(DEFAULT_CONTENT)
            .status(DEFAULT_STATUS)
            .owner(owner)
    );

    Tag foreignTag = new Tag()
        .tagName("foreign")
        .color("red")
        .owner(other);
```

```

em.persist(foreignTag);
em.flush();

NoteDTO noteDTO = noteMapper.toDto(insertedNote);

TagDTO foreignTagDTO = new TagDTO();
foreignTagDTO.setId(foreignTag.getId());
foreignTagDTO.setTagName(foreignTag.getTagName());
foreignTagDTO.setColor(foreignTag.getColor());

noteDTO.setTags(Set.of(foreignTagDTO));

restNoteMockMvc
    .perform(
        put(ENTITY_API_URL_ID, insertedNote.getId())
        .contentType(MediaType.APPLICATION_JSON)
        .content(om.writeValueAsBytes(noteDTO))
    )
    .andExpect(status().isBadRequest());

em.clear();
Note persisted = noteRepository.findById(insertedNote.getId()).orElseThrow();
assertThat(persisted.getTags()).isEmpty();
}

```

L'operazione di aggiornamento della nota con il tag "estraneo" viene eseguita tramite endpoint REST e il comportamento atteso è il rifiuto della richiesta con codice HTTP 400.

Fase Green: implementazione minima nel service layer

Per evitare di caricare inutilmente tutte le entità Tag, è stata aggiunta nella repository una query derivata che conta quanti tag, tra quelli passati in input, appartengono all'utente corrente.

Listing 2: Metodo di supporto aggiunto alla repository

```

public interface TagRepository extends JpaRepository<Tag, Long> {

    long countByIdInAndOwnerLogin(Collection<Long> ids, String login);

}

```

Nel service layer, il controllo può essere implementato come segue:

Listing 3: Implementazione minima del controllo nel service layer

```

@Service
@Transactional
public class NoteService {

    private final NoteRepository noteRepository;
    private final UserService userService;
    private final NoteMapper noteMapper;
    private final TagRepository tagRepository;

    public NoteService(
        NoteRepository noteRepository,
        NoteMapper noteMapper,
        UserService userService,
        TagRepository tagRepository
    ) {
        this.noteRepository = noteRepository;
        this.userService = userService;
        this.noteMapper = noteMapper;
        this.tagRepository = tagRepository;
    }

    public NoteDTO update(NoteDTO noteDTO) {
        validateOwnedTags(noteDTO);

        Note existingNote = noteRepository
            .findById(noteDTO.getId())
            .orElseThrow(() -> new IllegalArgumentException(
                "Note not found with id " + noteDTO.getId()
            ));

        noteMapper.partialUpdate(existingNote, noteDTO);
        existingNote = noteRepository.save(existingNote);
        return noteMapper.toDto(existingNote);
    }

    private void validateOwnedTags(NoteDTO noteDTO) {
        if (noteDTO.getTags() == null || noteDTO.getTags().isEmpty()) {
            return;
        }

        String currentLogin = SecurityUtils.getCurrentUserLogin()
            .orElseThrow(() -> new IllegalStateException("Current user not found"));
    }
}

```

```

Set<Long> tagIds = noteDT0.getTags().stream()
    .map(TagDT0::getId)
    .filter(Objects::nonNull)
    .collect(Collectors.toSet());

if (tagIds.size() != noteDT0.getTags().size()) {
    throw new BadRequestAlertException(
        "Invalid tag list",
        "note",
        "taginvalid"
    );
}

long ownedTagCount = tagRepository.countByIdInAndOwnerLogin(tagIds,
    ↪ currentLogin);

if (ownedTagCount != tagIds.size()) {
    throw new BadRequestAlertException(
        "A note can only use tags owned by the current user",
        "note",
        "foreignntag"
    );
}
}
}

```

L'utente autenticato viene recuperato tramite `SecurityUtils.getCurrentUserLogin()`, utility per ottenere il login associato al contesto di sicurezza corrente. In questo modo la regola di dominio resta centralizzata nel service layer e non dipende dal client.

Fase Refactor: consolidamento e pulizia

Una volta superato il test, il codice è stato rifinito separando chiaramente la logica di validazione dal resto del flusso di aggiornamento [?]. In particolare, il controllo è stato estratto nel metodo privato `validateOwnedTags(...)`, con il vantaggio di ridurre la duplicazione: lo stesso controllo può essere richiamato non solo in `update(...)`, ma anche in `save(...)` e in `partialUpdate(...)`.

4.3.3 *Testing della relazione molti-a-molti Note-Tag*

La relazione molti-a-molti tra Note e Tag rappresenta un punto sensibile nei test di integrazione, poiché coinvolge la join table generata automaticamente da

JPA/Hibernate, la gestione delle collezioni e la serializzazione degli oggetti tramite endpoint REST.

Dal punto di vista del testing, è importante verificare che:

- l'associazione tra note e tag venga salvata correttamente nella join table;
- uno stesso tag possa essere associato a più note;
- la rimozione di un tag da una nota aggiorni correttamente la relazione;
- la serializzazione JSON delle collezioni funzioni correttamente negli endpoint REST.

Il Listing 4 mostra un esempio di test di integrazione che verifica l'associazione di più tag a una singola nota.

Listing 4: Test di integrazione della relazione Note-Tag

```
@Test
@Transactional
void shouldAssociateMultipleTagsToNote() throws Exception {
    User user = getOrCreateUser(em, "user");

    insertedNote =
    noteRepository.saveAndFlush(
        new Note()
            .title("Test note")
            .content("content")
            .status(NoteStatus.NORMAL)
            .owner(user)
    );

    Tag tag1 = tagRepository.saveAndFlush(
        new Tag().tagName("tag1").color("blue").owner(user)
    );

    Tag tag2 = tagRepository.saveAndFlush(
        new Tag().tagName("tag2").color("red").owner(user)
    );

    NoteDTO dto = noteMapper.toDto(insertedNote);

    TagDTO t1 = new TagDTO();
    t1.setId(tag1.getId());
```



```

TagDTO t2 = new TagDTO();
t2.setId(tag2.getId());

dto.setTags(Set.of(t1, t2));

restNoteMockMvc.perform(
    put(ENTITY_API_URL_ID, insertedNote.getId())
    .contentType(MediaType.APPLICATION_JSON)
    .content(om.writeValueAsBytes(dto))
    ).andExpect(status().isOk());

em.clear();

Note persisted = noteRepository.findById(insertedNote.getId()).orElseThrow();

assertThat(persisted.getTags()).hasSize(2);
}

```

Il test crea una nota e due tag appartenenti allo stesso utente, invia una richiesta REST di aggiornamento della nota e verifica successivamente, interrogando il database, che la collezione dei tag associati contenga effettivamente due elementi. In questo modo si verifica implicitamente anche la corretta persistenza della join table generata da JPA.

4.4 UNIT TESTING FRONTEND CON JEST

Per il testing del frontend Angular è stato utilizzato Jest, framework di unit testing integrato nel progetto generato da JHipster. Jest consente di verificare in modo rapido e isolato il comportamento di componenti, servizi e classi di supporto del frontend, senza richiedere l'avvio completo dell'applicazione o l'interazione con un browser reale.

L'attenzione si concentra quindi sulla logica lato client, sulla gestione dello stato del componente e sulla corretta interazione con i servizi Angular. Nel progetto i test Jest sono collocati accanto ai file del frontend. Per l'entità Note, ad esempio, JHipster ha generato automaticamente test per:

- il componente di lista `note.component.spec.ts`;
- il service REST `note.service.spec.ts`;
- il servizio di gestione del form `note-form.service.spec.ts`;
- il componente di aggiornamento `note-update-dialog.component.spec.ts`.

4.4.1 Esempi di test generati

JHipster genera verifiche per le principali operazioni CRUD lato client: ricerca di un elemento, creazione, aggiornamento completo, aggiornamento parziale, recupero della lista ed eliminazione. I test utilizzano `HttpTestingController` per simulare le richieste HTTP e verificare che il service costruisca correttamente le chiamate REST e trasformi i dati restituiti dal backend nel modello `TypeScript` usato dal frontend.

Listing 5: Estratto semplificato di test Jest per il service delle note

```

1 describe('Note Service', () => {
2   let service: NoteService;
3   let httpMock: HttpTestingController;
4   let expectedResult: INote | INote[] | boolean | null;
5
6   beforeEach(() => {
7     TestBed.configureTestingModule({
8       providers: [provideHttpClient(), provideHttpClientTesting()],
9     });
10    service = TestBed.inject(NoteService);
11    httpMock = TestBed.inject(HttpTestingController);
12  });
13
14  it('should find an element', () => {
15    service.find(123).subscribe(resp => (expectedResult = resp.body));
16
17    const req = httpMock.expectOne({ method: 'GET' });
18    req.flush(returnedFromService);
19
20    expect(expectedResult).toMatchObject(expected);
21  });
22
23  it('should delete a Note', () => {
24    service.delete(123).subscribe(resp => (expectedResult = resp.ok));
25
26    const req = httpMock.expectOne({ method: 'DELETE' });
27    req.flush({ status: 200 });
28
29    expect(expectedResult).toBe(true);
30  });
31 });

```

4.5 BEHAVIOR-DRIVEN DEVELOPMENT CON CUCUMBER

Per formalizzare comportamenti ad alto livello è possibile integrare Cucumber, framework che consente di descrivere scenari di utilizzo attraverso il linguaggio Gherkin. Questo approccio permette di esprimere i requisiti funzionali in una forma leggibile anche da soggetti non tecnici, favorendo la comunicazione tra sviluppatori e stakeholder.

Un esempio di scenario sviluppato nel caso di studio è riportato nel Listing 6.

Listing 6: Esempio di scenario Gherkin

Feature: Gestione delle note

Scenario: Creazione di una nota con due tag

Given l'utente autenticato ha creato i tag "Studio" e "Lavoro"

When crea una nota con titolo "Preparare esame"

And associa i tag "Studio" e "Lavoro"

Then la nota deve essere salvata con 2 tag associati

Gli scenari Gherkin vengono poi collegati al codice applicativo tramite le *step definitions*, implementate in Java e integrate con il contesto Spring Boot. Ogni passo descritto nello scenario viene associato a un metodo che esegue le operazioni necessarie e verifica gli effetti prodotti sul sistema. Le step definitions possono interagire con i service applicativi oppure con gli endpoint REST, a seconda del livello di astrazione scelto per il test.

Listing 7: Esempio semplificato di step definition

```
@Transactional
@WithMockUser(username = "user")
public class GestioneNoteStepDefs extends StepDefs {

    @Autowired
    private EntityManager em;

    @Autowired
    private TagRepository tagRepository;

    @Autowired
    private NoteRepository noteRepository;

    private Note notaCreata;
    private final Set<Tag> tagDaAssociare = new HashSet<>();
```

```

@Before
public void cleanScenarioState() {
    notaCreata = null;
    tagDaAssociare.clear();
}

@Given("l'utente autenticato ha creato i tag {string} e {string}")
public void utenteHaCreatoITag(String nomeTag1, String nomeTag2) {
    Tag tag1 = creaTagSeAssente(nomeTag1);
    Tag tag2 = creaTagSeAssente(nomeTag2);

    tagDaAssociare.add(tag1);
    tagDaAssociare.add(tag2);
}

@When("crea una nota con titolo {string}")
public void creaUnaNotaConTitolo(String titolo) {
    User owner = getOrCreateUser("user");

    Note note = new Note()
        .title(titolo)
        .content("contenuto di test")
        .status(NoteStatus.NORMAL)
        .owner(owner);

    notaCreata = noteRepository.saveAndFlush(note);
}
}

```

JHipster, tuttavia, non genera molto oltre la configurazione base di Cucumber. Nel caso in esame, il framework si limita essenzialmente a fornire uno scenario di esempio, riportato nel Listing 8.

Feature: User management

Scenario: Retrieve administrator user

When I search user 'admin'

Then the user is found

And his last name is 'Administrator'

Listing 8: scenario generato

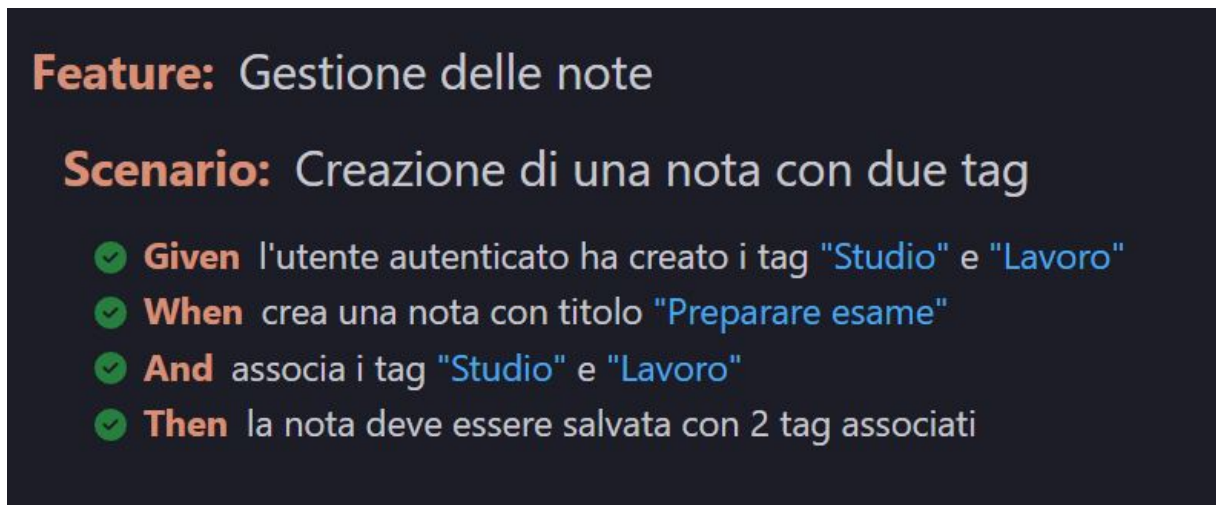


Figura 6: report di cucumber.

4.6 END-TO-END TESTING CON CYPRESS

Per testare l'applicazione dal punto di vista dell'utente finale viene integrato Cypress, uno strumento dedicato ai test end-to-end per applicazioni web. Cypress consente di eseguire il browser in modalità controllata e simulare interazioni reali con l'interfaccia Angular, verificando non solo la correttezza delle singole chiamate HTTP, ma soprattutto il comportamento complessivo dell'applicazione durante l'interazione utente [5].

I test Cypress auto generati dal progetto coprono già diversi aspetti dell'applicazione, in particolare:

- autenticazione dell'utente;
- navigazione tra le pagine;
- accesso alle pagine CRUD delle entità;

I test generati automaticamente mostrano già la struttura generale degli scenari E2E: apertura della pagina dell'entità, accesso al form di creazione, visualizzazione della pagina di dettaglio, modifica e ritorno alla lista.

Tuttavia, nel caso di studio considerato, l'interfaccia relativa alle note è stata personalizzata rispetto al CRUD standard generato da JHipster. Per questo motivo i test pregenerati relativi all'entità Note non risultano aderenti al comportamento reale della UI e sono stati quindi adattati.

In particolare, sono stati implementati questi test specifici per verificare tre scenari applicativi:

- creazione di una nuova nota e verifica della sua comparsa nella lista;
- creazione di una nota con associazione immediata di un tag;
- apertura del dialog di modifica di una nota esistente e aggiunta di un ulteriore tag, con verifica dell'aggiornamento nell'interfaccia.

Il Listing 9 mostra un estratto di un test Cypress. Lo scenario esegue l'accesso alla pagina delle note, crea una nuova nota, apre il dialog di aggiornamento tramite selezione della card corrispondente e aggiunge un nuovo tag, verificando sia la corretta risposta HTTP sia l'effettiva presenza dei tag nella lista delle note.

Listing 9: Test end-to-end con Cypress per creazione e aggiornamento di una nota

```

1  it('should create a note, open update dialog, add a tag and show both in UI', ()
   ↪   => {
2      const title = `E2E dialog tag note ${Date.now()}`;
3      const content = 'Nota da aggiornare dal dialog';
4      const initialTag = `iniziale-${Date.now()}`;
5      const addedTag = `aggiunto-${Date.now()}`;
6
7      cy.visit('/note');
8      cy.wait('@getNotes');
9
10     cy.get('[data-cy="title"]').clear().type(title);
11     cy.get('[data-cy="content"]').clear().type(content);
12     cy.get('[data-cy="tagInput"]').first().type(`${initialTag}{enter}`);
13     cy.get('[data-cy="NoteSaveButton"]').click();
14
15     cy.wait('@postNote').then(({ response }) => {
16         expect(response?.statusCode).to.eq(201);
17     });
18
19     cy.contains('[data-cy="noteTitle"]', title).should('be.visible');
20
21     cy.contains('[data-cy="noteCard"] [data-cy="noteTitle"]', title)
22       .closest('[data-cy="noteCard"]')
23       .click();
24
25     cy.get('[data-cy="noteUpdateDialog"]')
26       .should('be.visible')
27       .within(() => {
28         cy.contains('[data-cy="tagChip"]', initialTag).should('be.visible');
29         cy.get('[data-cy="tagInput"]').type(`${addedTag}{enter}`);

```

```

30     cy.get('[data-cy="NoteSaveEdit"]').click({ force: true });
31   });
32
33   cy.wait('@patchNote').then(({ response }) => {
34     expect(response?.statusCode).to.eq(200);
35   });
36
37   cy.contains('[data-cy="noteCard"] [data-cy="noteTitle"]', title)
38     .closest('[data-cy="noteCard"]')
39     .within(() => {
40       cy.contains('[data-cy="tagChip"]', initialTag).should('be.visible');
41       cy.contains('[data-cy="tagChip"]', addedTag).should('be.visible');
42     });
43   });

```

i test Cypress possono essere lanciati sia in ambiente di sviluppo sia in modalità automatizzata. Durante lo sviluppo conviene utilizzare l'interfaccia grafica di Cypress, che consente di osservare passo per passo i comandi eseguiti, il DOM renderizzato e le chiamate HTTP intercettate. In fase di esecuzione automatizzata, invece, la suite può essere avviata in modalità *headless*, adatto alle pipeline CI.

4.7 MUTATION TESTING CON PIT

Per valutare non solo la copertura del codice ma anche l'effettiva capacità dei test di individuare errori nel comportamento del sistema, è possibile introdurre il *mutation testing* tramite lo strumento PIT (*Pitest*).

A differenza della semplice copertura del codice, il mutation testing introduce modifiche artificiali nel codice sorgente, chiamate *mutanti*, per verificare se i test sono in grado di individuare tali variazioni. Se un mutante viene rilevato da almeno un test, esso viene considerato "ucciso"; in caso contrario, indica una possibile debolezza nella suite di test [3, 20].

Per integrare PIT con JHipster basterà aggiungere il plugin con maven tramite la configurazione del pom.xml. come mostrato nel Listing 10.

Listing 10: Configurazione Maven del plugin PIT

```

<plugin>
  <groupId>org.pitest</groupId>
  <artifactId>pitest-maven</artifactId>
  <version>${pitest.version}</version>
  <dependencies>
    <dependency>

```

```

    <groupId>org.pitest</groupId>
    <artifactId>pitest-junit5-plugin</artifactId>
    <version>${pitest-junit5.version}</version>
  </dependency>
</dependencies>
<configuration>
  <targetClasses>
    <param>com.giovannip.*</param>
  </targetClasses>
  <targetTests>
    <param>com.giovannip.*</param>
  </targetTests>
  <excludedClasses>
    <param>com.giovannip.noticeme.service.mapper.*MapperImpl</param>
  </excludedClasses>
  <excludedTestClasses>
    <param>com.giovannip.noticeme.TechnicalStructureTest</param>
    <param>com.giovannip.noticeme.*Architecture*</param>
    <param>com.giovannip.noticeme.*ArchUnit*</param>
    <param>com.giovannip.noticeme.*Structure*</param>
    <param>com.giovannip.noticeme.cucumber.*</param>
    <param>gatling.simulations.*</param>
  </excludedTestClasses>
  <mutationThreshold>1</mutationThreshold>
  <avoidCallsTo>
    <param>java.util.logging</param>
    <param>org.slf4j</param>
  </avoidCallsTo>
  <threads>4</threads>
  <outputFormats>
    <param>HTML</param>
  </outputFormats>
</configuration>
</plugin>

```

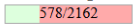
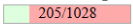
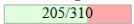
L'esecuzione dell'analisi avviene tramite il comando seguente; naturalmente, PIT è in grado di produrre un risultato significativo solo se la suite di test viene eseguita senza errori:

```
./mvnw org.pitest:pitest-maven:mutationCoverage
```

Al termine dell'esecuzione, PIT genera automaticamente un report HTML contenente statistiche dettagliate sul numero di mutanti generati, sul numero di mutanti uccisi

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
67	27% 	20% 	66% 

Breakdown by Package

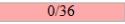
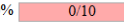
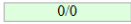
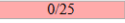
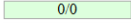
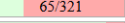
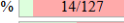
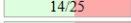
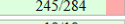
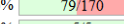
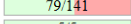
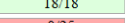
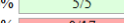
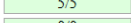
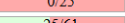
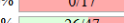
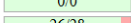
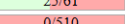
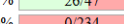
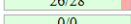
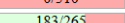
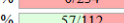
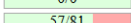
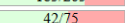
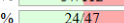
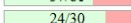
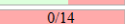
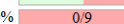
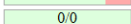
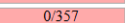
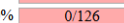
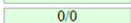
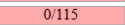
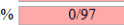
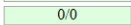
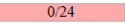
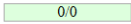
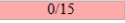
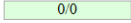
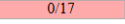
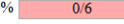
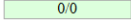
Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
com.giovannip.noticeme	2	0% 	0% 	100% 
com.giovannip.noticeme.aop.logging	1	0% 	0% 	100% 
com.giovannip.noticeme.config	16	20% 	11% 	56% 
com.giovannip.noticeme.domain	7	86% 	46% 	56% 
com.giovannip.noticeme.management	1	100% 	100% 	100% 
com.giovannip.noticeme.repository	2	0% 	0% 	100% 
com.giovannip.noticeme.security	3	41% 	55% 	93% 
com.giovannip.noticeme.service	6	0% 	0% 	100% 
com.giovannip.noticeme.service.dto	8	69% 	51% 	70% 
com.giovannip.noticeme.service.mapper	3	56% 	51% 	80% 
com.giovannip.noticeme.web.filter	1	0% 	0% 	100% 
com.giovannip.noticeme.web.rest	9	0% 	0% 	100% 
com.giovannip.noticeme.web.rest.errors	3	0% 	0% 	100% 
com.giovannip.noticeme.web.rest.vm	3	0% 	0% 	100% 
com.giovannip.noticeme.web.websocket	1	0% 	0% 	100% 
com.giovannip.noticeme.web.websocket.dto	1	0% 	0% 	100% 

Figura 7: Panoramica del report generato da Pitest.

dai test e sui mutanti sopravvissuti. Il report permette inoltre di analizzare a livello di singola classe quali mutazioni non vengono rilevate dalla suite di test.

La Figura 7 mostra un esempio della schermata principale del report generato da PIT. I risultati ottenuti nel caso di studio mostrano che, nonostante una buona copertura strutturale del codice evidenziata dai report JaCoCo, una parte significativa dei mutanti sopravvive all'esecuzione dei test. Questo risultato è coerente con la natura della suite di test del progetto, composta in larga misura da test di integrazione orientati alla verifica del comportamento dell'applicazione tramite API REST. Sebbene tali test coprano ampie porzioni di codice, nella configurazione generata da JHipster essi non contribuiscono in modo efficace all'analisi di mutation testing condotta da PIT, che in questo contesto si focalizza sui soli test unitari.

4.8 PERFORMANCE TESTING CON GATLING

JHipster può integrare automaticamente Gatling come strumento per l'esecuzione di test di performance sulle API REST. Gatling consente di simulare un numero elevato di utenti concorrenti e di raccogliere metriche quali tempo medio di risposta, throughput e percentili di latenza [9].

Nel progetto generato, Gatling è integrato tramite dipendenze Maven e tramite il plugin `gatling-maven-plugin`, che consente di eseguire le simulazioni direttamente

durante la fase di testing del progetto.

Le simulazioni Gatling generate automaticamente da JHipster si trovano nel package `gatling.simulations`. Per ogni entità REST esposta dall'applicazione viene creata una simulazione che verifica il comportamento delle principali operazioni CRUD sotto carico.

4.8.1 *Struttura dei test pregenerati*

La simulazione `NoteGatlingTest` esegue una sequenza di richieste HTTP che riproduce un utilizzo realistico dell'API dell'applicazione.

Lo scenario segue la sequenza:

- tentativo di accesso senza autenticazione;
- autenticazione tramite endpoint `/api/authenticate`;
- verifica dell'accesso autenticato all'endpoint `/api/account`;
- esecuzione delle principali operazioni CRUD sull'entità `Note`.

Il Listing 11 mostra un estratto dello scenario principale della simulazione.

Listing 11: Estratto della simulazione Gatling per l'entità `Note`

```
ChainBuilder scn =
exec(http("First unauthenticated request")
.get("/api/account")
.check(status().is(401)))

.exec(http("Authentication")
.post("/api/authenticate")
.body(StringBody("{\"username\":\"admin\",\"password\":\"admin\"}"))
.check(header("Authorization").saveAs("access_token")))

.exec(http("Authenticated request")
.get("/api/account")
.check(status().is(200)))

.exec(http("Get all notes")
.get("/api/notes"))

.exec(http("Create new note")
.post("/api/notes"));
```

```

exec(http("Get created note")
.get("#{new_note_url}")

.exec(http("Delete created note")
.delete("#{new_note_url}")

```

Questo scenario consente di verificare non solo la correttezza delle risposte HTTP, ma anche il comportamento delle API in presenza di richieste concorrenti.

4.8.2 Simulazione del carico

Il carico viene introdotto tramite la strategia `rampUsers`, che simula un aumento progressivo degli utenti virtuali.

Listing 12: Configurazione dell'iniezione di utenti virtuali

```

setUp(
users.injectOpen(
rampUsers(Integer.getInteger("users", 100))
.during(Duration.ofMinutes(Integer.getInteger("ramp", 1)))
)
);

```

Nella configurazione predefinita, Gatling simula 100 utenti virtuali distribuiti nell'arco di 1 minuto. Questo approccio consente di osservare come il sistema reagisce all'aumento progressivo del carico, evitando picchi improvvisi di richieste. I parametri possono essere modificati durante l'esecuzione del test, permettendo di adattare facilmente il carico simulato. Per eseguire i test Gatling è necessario che l'applicazione sia già in esecuzione su una porta raggiungibile; per questo motivo tali test risultano particolarmente adatti a essere eseguiti in ambienti di staging o di integrazione continua, piuttosto che durante lo sviluppo locale.

```
./mvnw gatling:test -Dusers=200 -Dramp=2
```

Al termine dell'esecuzione Gatling genera automaticamente un report HTML contenente le principali metriche di performance come si può vedere nella Figura 8.

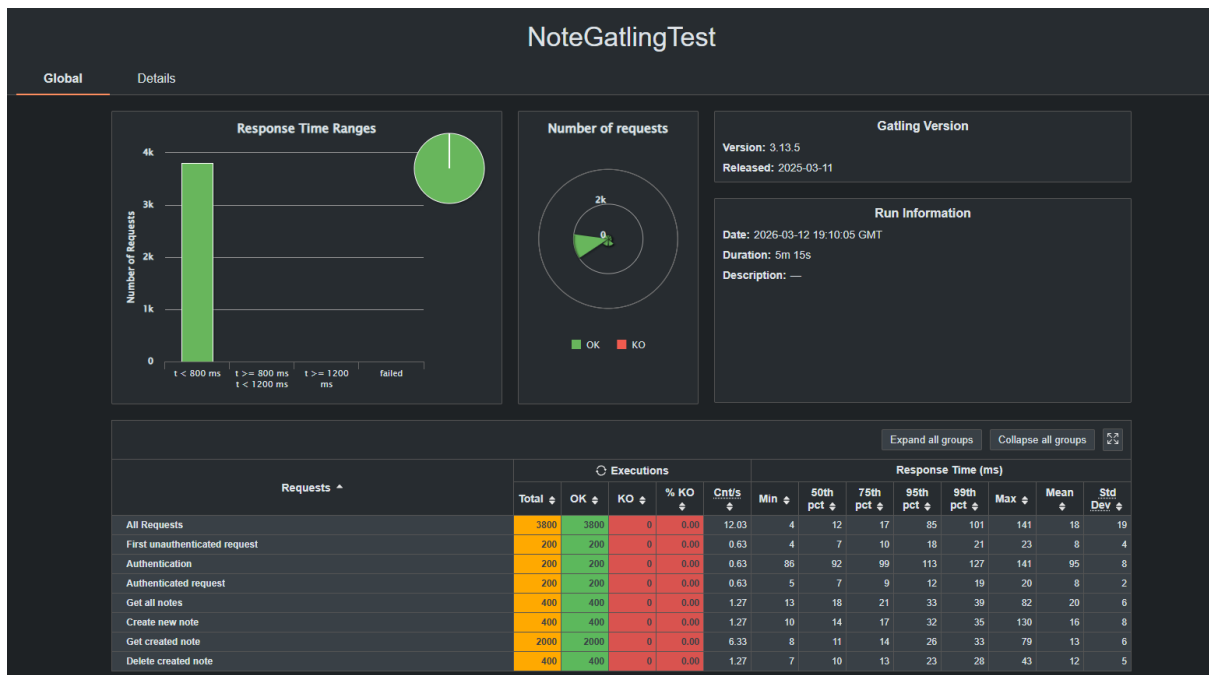


Figura 8: Panoramica del report di performance generato da Gatling al termine della simulazione.

4.8.3 Analisi dei report di performance

I report generati da Gatling includono informazioni quali:

- numero totale di richieste
- percentuale di richieste riuscite e fallite
- tempo minimo, medio e massimo di risposta
- percentili di latenza (quanto velocemente risponde il sistema nella maggior parte dei casi)
- throughput nel tempo (numero di richieste gestite dal sistema per secondo)

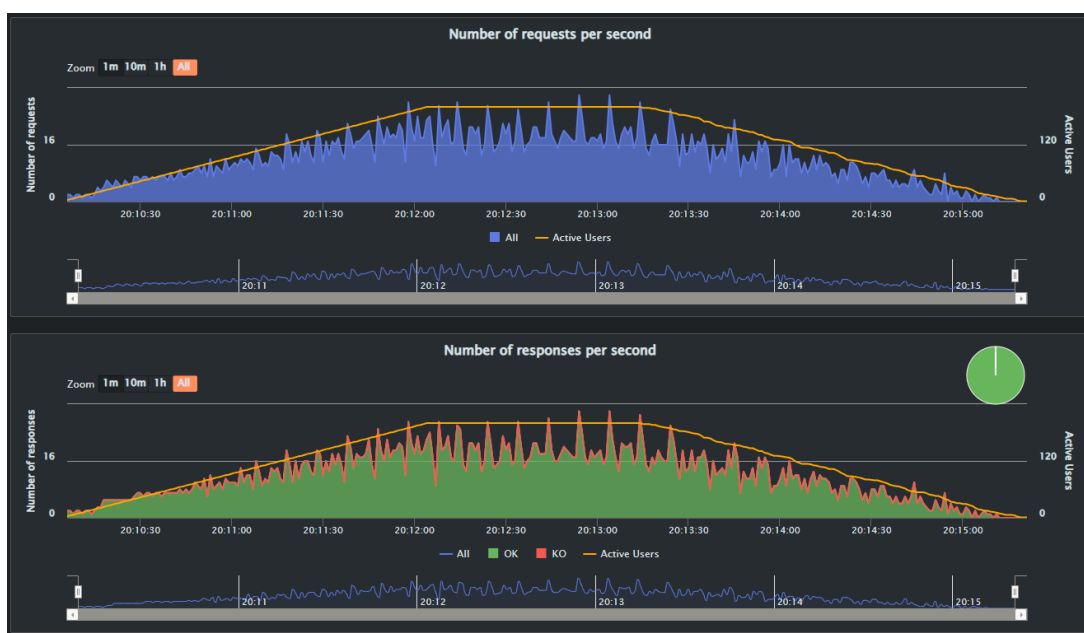


Figura 10: Throughput del sistema espresso come numero di richieste al secondo durante la simulazione.

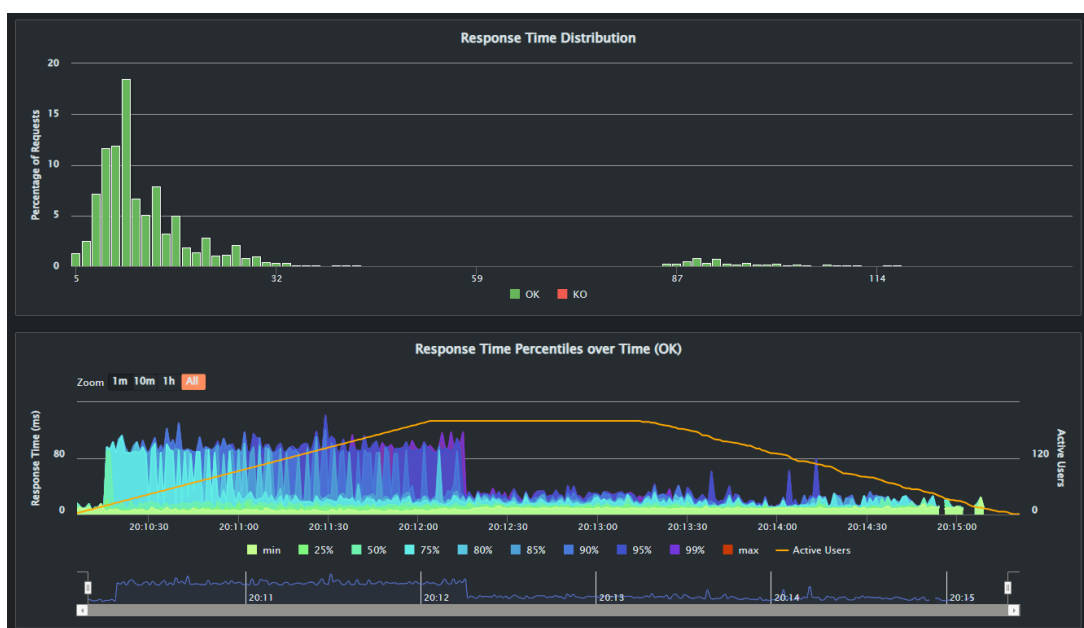


Figura 9: Distribuzione dei tempi di risposta durante l'esecuzione del test di carico.

Questi grafici consentono di identificare eventuali colli di bottiglia nelle API e di valutare la scalabilità del sistema sotto carico.

4.8.4 *Personalizzazione degli scenari di test*

I test generati automaticamente rappresentano una base iniziale per il performance testing. In contesti reali, tuttavia, è spesso necessario personalizzare gli scenari per riflettere in modo più accurato i comportamenti degli utenti.

Tra le principali estensioni possibili vi sono la simulazione di diversi profili di utenti (ad esempio utenti anonimi e autenticati, admin e normali), diverse sequenze di chiamate API che emulano diversi comportamenti sull'app, aumento progressivo del numero di utenti virtuali per valutare la scalabilità del sistema.

Gatling supporta inoltre strategie di iniezione più avanzate, come `constantUsersPerSec` o `stressPeakUsers`, che permettono di simulare scenari di carico più complessi.

4.8.5 *Ruolo del performance testing nel contesto della tesi*

Nel contesto della presente tesi i test di performance non costituiscono il focus principale dell'analisi, che è invece orientata alle pratiche di testing automatizzato e Continuous Integration.

Tuttavia, la presenza di Gatling all'interno dello stack JHipster dimostra come il framework favorisca un processo di verifica della qualità del software più ampio, includendo anche la validazione di requisiti non funzionali quali prestazioni e scalabilità.

INTEGRAZIONE CONTINUA NEL CASO DI STUDIO

Questo capitolo descrive l'infrastruttura di Continuous Integration e le scelte effettuate per automatizzare build, test e analisi di qualità del progetto. La soluzione adottata consiste in un ambiente CI locale basato su Jenkins e Docker, integrato con SonarQube, un registry Docker locale e gli strumenti di test discussi nei capitoli precedenti.

JHipster mette a disposizione configurazioni pronte per pipeline CI attraverso il CLI (ad esempio per GitHub Actions, GitLab CI o Jenkins). Tuttavia, nel caso di studio tali configurazioni non risultavano pienamente compatibili con le esigenze del progetto; per questo motivo è stato realizzato un `Jenkinsfile` personalizzato, affiancato da un file `docker-compose` per la creazione di una pipeline CI locale [15, 14, 6].

5.1 INFRASTRUTTURA CI LOCALE BASATA SU DOCKER

Per rendere l'ambiente facilmente riproducibile, i servizi necessari alla pipeline sono stati orchestrati tramite `docker-compose`.

L'infrastruttura utilizzata nel caso di studio comprende i seguenti componenti principali:

- Jenkins, responsabile dell'esecuzione della pipeline;
- SonarQube, utilizzato per l'analisi statica del codice e la valutazione dei quality gate;
- un database PostgreSQL dedicato al funzionamento di SonarQube;
- un registry Docker locale per la pubblicazione delle immagini generate.

Il Listing 13 mostra un estratto della configurazione `docker-compose`. la configurazione completa è disponibile nel repository del progetto.

Listing 13: Estratto del `docker-compose` per l'ambiente CI locale

networks:

```

ci-net:

services:
  sonar-db:
    image: postgres:16
    environment:
      POSTGRES_USER: sonar
      POSTGRES_PASSWORD: sonar
      POSTGRES_DB: sonar

  sonarqube:
    image: sonarqube:community
    depends_on:
      - sonar-db
    ports:
      - "19000:9000"

  registry:
    image: registry
    ports:
      - "15000:5000"

  jenkins:
    image: jenkins/jenkins:lts
    user: root
    ports:
      - "18080:8080"
      - "50000:50000"
    depends_on:
      - sonarqube
      - registry

```

5.2 PIPELINE JENKINS: STRUTTURA E STRATEGIA MULTI-BRANCH

La pipeline è stata implementata tramite Jenkinsfile dichiarativo. La strategia adottata differenzia le attività in base al tipo di branch e al contesto di esecuzione:

- sulle *pull request* viene eseguita una suite rapida di unit test;
- sui branch *feature/** viene eseguito un controllo qualità intermedio con raccolta di metriche;
- sul branch *dev* viene eseguita una suite completa, comprensiva di report Cucumber e, opzionalmente, di test più onerosi;

- sul branch main viene eseguita la build di rilascio e il packaging per la produzione.

Questa strategia consente, da un lato, di ottenere feedback rapido durante lo sviluppo e, dall'altro, di eseguire verifiche più approfondite prima dell'integrazione definitiva del codice.

5.2.1 Validazione rapida su Pull Request

Per le pull request la pipeline esegue una suite rapida di test, con l'obiettivo di fornire feedback immediato prima del merge.

Listing 14: Estratto Jenkinsfile: validazione PR

```
stage('pr-validation') {
  when { changeRequest() }
  steps {
    sh './mvnw -ntp -Pdev,webapp,no-liquibase,frontend-test clean test'
  }
  post {
    always {
      junit allowEmptyResults: true, testResults:
        ↪ '**/target/surefire-reports/*.xml'
    }
  }
}
```

5.2.2 Suite completa su branch dev e raccolta report

Sul branch dev viene eseguita una suite più completa, includendo test di integrazione e la generazione di report. vengono raccolti report JUnit (surefire) e (failsafe), vengono pubblicati report di code coverage tramite JaCoCo, vengono pubblicati report Cucumber quando presenti.

Listing 15: Estratto Jenkinsfile: suite completa su dev e report

```
stage('dev-full-qa') {
  when { branch 'dev' }
  steps {
    sh './mvnw -ntp -Pdev,webapp,frontend-test verify'
  }
  post {
    always {
      cucumber fileIncludePattern: '**/cucumber-reports/*.json'
    }
  }
}
```

```

junit allowEmptyResults: true, testResults:
  ↪ '**/target/surefire-reports/*.xml'
junit allowEmptyResults: true, testResults:
  ↪ '**/target/failsafe-reports/*.xml'
publishHTML(target: [
  reportDir: 'target/site/jacoco',
  reportFiles: 'index.html',
  reportName: 'Jacoco Coverage'
])
}
}
}

```

5.2.3 Test ad alto costo computazionale

Alcuni strumenti di verifica della qualità, come il Mutation Testing e i test di performance, richiedono tempi di esecuzione significativamente più lunghi rispetto ai test tradizionali. Per questo motivo tali verifiche vengono eseguite solo in condizioni specifiche.

Nel caso di studio, l'esecuzione dei test ad alto costo computazionale può essere attivata in due modi:

- manualmente tramite il parametro della pipeline RUN_HEAVY;
- automaticamente se il messaggio di commit contiene tag specifici come [heavy] o [perf].

Listing 16: Estratto Jenkinsfile: PIT e Gatling condizionali

```

stage('Heavy Tests (optional)') {
  when {
    allOf {
      branch 'dev'
      anyOf {
        expression { params.RUN_HEAVY }
        expression { env.COMMIT_MSG.contains('[heavy]') }
        expression { env.COMMIT_MSG.contains('[perf]') }
      }
    }
  }
  steps {
    sh './mvnw org.pitest:pitest-maven:mutationCoverage'
  }
}

```

```

sh './mvnw -ntp -Pdev,webapp -DskipTests package'
sh '''
    nohup java -jar target/*.jar > app.log 2>&1 &
    echo $! > app.pid
'''

sh '''
    for i in $(seq 1 60); do
        if curl -fsS http://localhost:8080/management/health | grep -q
        ↪ '"status":"UP"'; then
            echo "Application is UP"
            exit 0
        fi
        sleep 5
    done
    echo "Application failed to start"
    tail -100 app.log || true
    exit 1
'''
sh './mvnw gatling:test'
}
post {
    always {
        sh '''
            if [ -f app.pid ]; then
                kill $(cat app.pid) || true
            fi
            '''
        archiveArtifacts artifacts: 'target/pit-reports/**', fingerprint: true
        archiveArtifacts artifacts: 'target/gatling/**', fingerprint: true
    }
}
}

```

5.2.4 Analisi statica e Quality Gate con SonarQube

Per la valutazione automatica della qualità del codice è stato integrato SonarQube all'interno della pipeline Jenkins. L'analisi statica viene eseguita sul branch dev e utilizza i report di copertura generati da JaCoCo [23].

Al termine dell'analisi viene valutato il *Quality Gate*, che rappresenta un insieme di criteri minimi di qualità (ad esempio copertura del codice, presenza di bug o code smells). Nel caso in cui il Quality Gate non venga superato, la pipeline viene interrotta.

Listing 17: Estratto Jenkinsfile: analisi SonarQube

```

stage('Sonar Analysis') {
    when { branch 'dev' }
    steps {
        withSonarQubeEnv('sonar') {
            script {
                def reports = 'target/site/jacoco/jacoco.xml'
                if (fileExists('target/pit-reports/jacoco.xml')) {
                    reports += ',target/pit-reports/jacoco.xml'
                }
                sh """
                ./mvnw sonar:sonar \
                -Dsonar.coverage.jacoco.xmlReportPaths=\${reports}
                """
            }
        }
    }
}

```

Listing 18: Estratto Jenkinsfile: verifica del Quality Gate

```

stage('Quality Gate') {
    when { branch 'dev' }
    steps {
        timeout(time: 20, unit: 'MINUTES') {
            waitForQualityGate abortPipeline: true
        }
    }
}

```

Al termine dell'analisi, SonarQube mette a disposizione una dashboard che sintetizza lo stato qualitativo del progetto attraverso diverse metriche. Tra queste risultano particolarmente rilevanti l'esito del *Quality Gate*, la copertura del codice e il numero di problemi individuati dall'analisi statica.

La Figura 11 mostra la pagina principale del progetto su SonarQube, nella quale è possibile osservare immediatamente l'esito del *Quality Gate*. La Figura 12 riporta invece la sezione *Overview*, che raccoglie le principali metriche di qualità del progetto. Infine, la Figura 13 mostra la lista dei problemi individuati dall'analisi statica, tra cui *code smells* e potenziali bug.

5.2 PIPELINE JENKINS: STRUTTURA E STRATEGIA MULTI-BRANCH

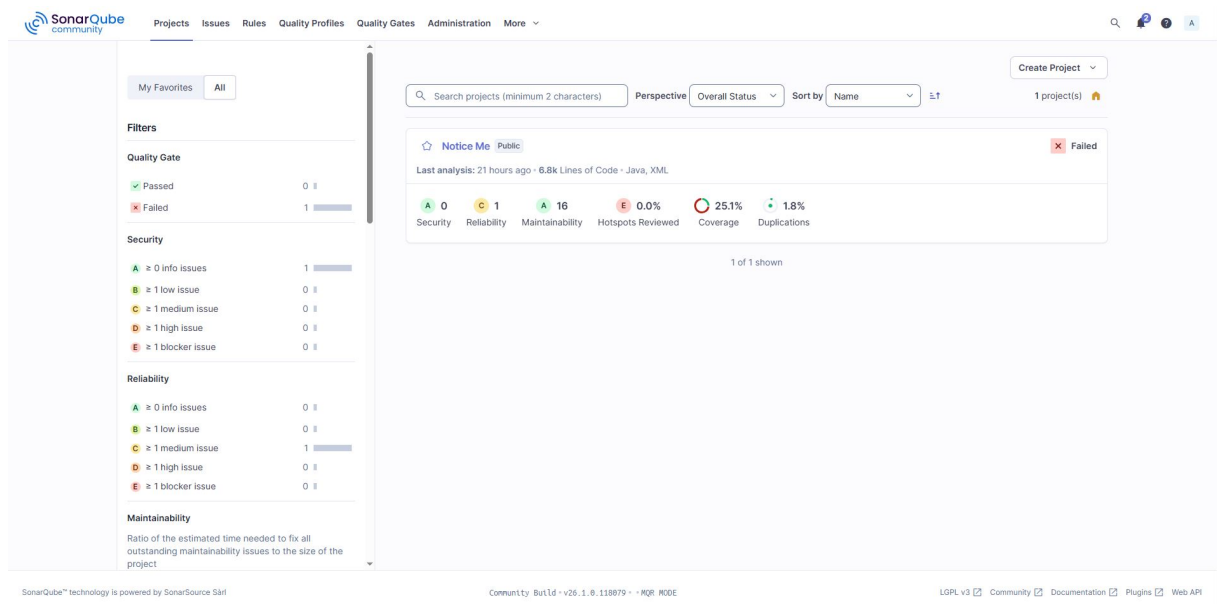


Figura 11: Pagina principale del progetto su SonarQube con l'esito del Quality Gate.

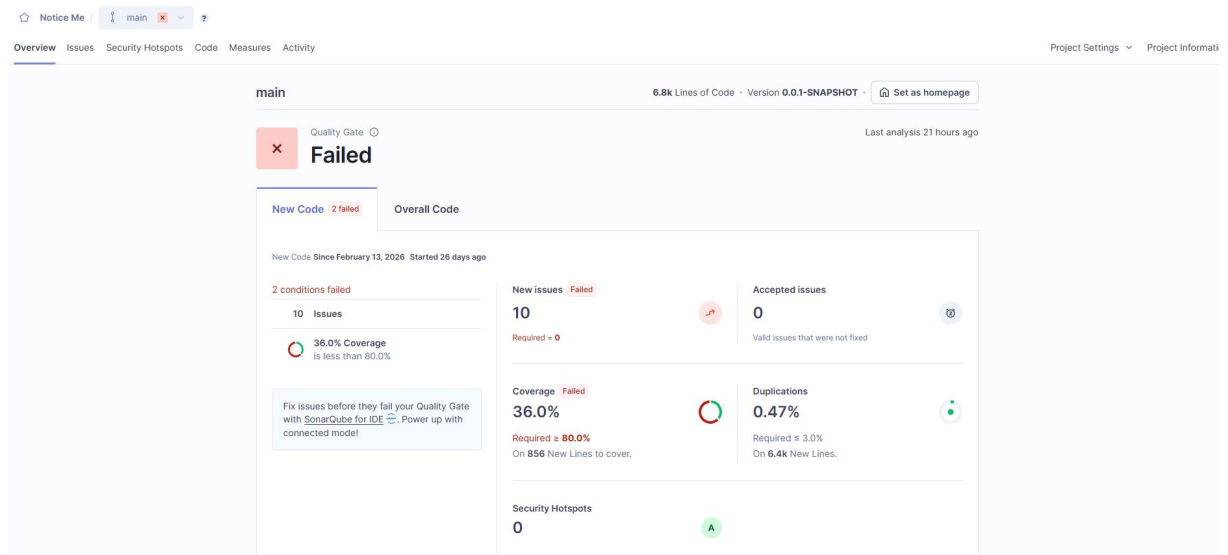


Figura 12: Vista Overview di SonarQube con le principali metriche di qualità del progetto.

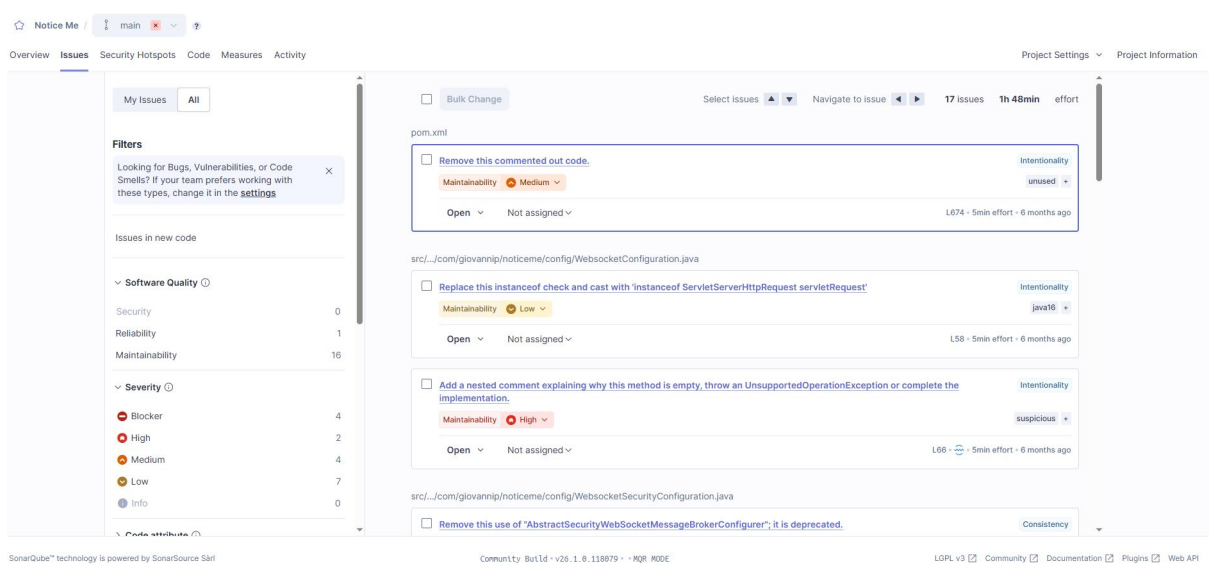


Figura 13: Sezione Issues di SonarQube con i problemi individuati dall'analisi statica del codice.

5.2.5 Build di rilascio e packaging in produzione

Sul branch main la pipeline esegue una build di produzione e produce un'immagine Docker tramite `jib:build` [11] per costruire l'immagine senza richiedere un Dockerfile esplicito per il backend, pubblicandola su un registry locale.

Listing 19: Estratto Jenkinsfile: build di rilascio e pubblicazione immagine

```
stage('Release Build') {
  when { branch 'main' }
  steps {
    sh './mvnw -ntp -Pprod clean package -DskipTests'
    sh """
      ./mvnw -Pprod jib:build \
      -Djib.to.image=localhost:15000/noticeme:${env.GIT_COMMIT}
    """
  }
}
```

Come fare deploy

Una volta pubblicata l'immagine nel registry, il deploy può essere eseguito in modo relativamente semplice sul nodo di destinazione. il server che ospita l'applicazione scarica l'immagine dal registry e avvia un nuovo container passando la configurazione

necessaria, ad esempio il profilo prod e i parametri di connessione al database MySQL. In uno scenario semplificato, il deploy consiste quindi in tre passaggi: recupero dell'immagine, avvio del container e verifica del corretto avvio del servizio.

Un possibile comando di esecuzione è il seguente:

```
docker pull localhost:15000/noticeme:<tag>
```

```
docker run -d \  
--name noticeme-app \  
-p 8080:8080 \  
-e SPRING_PROFILES_ACTIVE=prod \  
-e SPRING_DATASOURCE_URL=jdbc:mysql://mysql:3306/noticeme \  
-e SPRING_DATASOURCE_USERNAME=noticeme \  
-e SPRING_DATASOURCE_PASSWORD=noticeme \  
localhost:15000/noticeme:<tag>
```

Listing 20: Esempio semplificato di deploy del container

VALUTAZIONE CRITICA DEL CASO DI STUDIO

Questo capitolo interpreta criticamente i risultati emersi dal caso di studio con l'obiettivo di rispondere alla domanda di ricerca della tesi: in che misura un framework di generazione full-stack come JHipster supporta concretamente l'adozione di metodologie orientate alla qualità del software, in particolare Test-Driven Development (TDD), Behavior-Driven Development (BDD) e Continuous Integration (CI)?

L'analisi condotta nei capitoli precedenti mostra che JHipster non impone una metodologia di sviluppo specifica, ma fornisce una base architetturale e tecnologica che facilita l'introduzione di pratiche di testing multilivello e di automazione del ciclo di build. Allo stesso tempo, l'adozione effettiva di tali pratiche richiede una comprensione del funzionamento dei framework sottostanti e delle tecnologie DevOps coinvolte.

6.1 LIMITI DELLO STUDIO E MINACCE ALLA VALIDITÀ

Come in ogni studio empirico, i risultati ottenuti devono essere interpretati considerando alcune possibili minacce alla validità di esso.

- **Validità esterna.** Il progetto sviluppato rappresenta un caso di studio di complessità intermedia e utilizza una configurazione specifica di JHipster: architettura monolitica, frontend Angular, autenticazione JWT e database SQL. Configurazioni alternative (ad esempio architetture a microservizi o frontend differenti) potrebbero produrre risultati diversi.
- **Validità costruttiva.** La valutazione si basa su metriche quantitative come code coverage, mutation score e indicatori di qualità statica del codice. Sebbene tali metriche forniscano evidenze utili, esse non esauriscono il concetto di qualità del software e non misurano direttamente aspetti come la comprensibilità del codice o la qualità del design architetturale.
- **Validità interna.** Il caso di studio è stato sviluppato individualmente. Questo semplifica l'analisi del processo di sviluppo, ma limita la possibilità di osservare

pienamente alcuni benefici delle pratiche collaborative, in particolare quelli associati al BDD e alla Continuous Integration in team distribuiti.

- **Ambiente sperimentale.** La pipeline CI è stata realizzata in un ambiente locale basato su Docker, Jenkins e SonarQube. Sebbene ciò renda il sistema facilmente riproducibile, un'infrastruttura cloud o aziendale potrebbe introdurre ulteriori complessità operative.

Nonostante queste limitazioni, il caso di studio fornisce evidenze concrete su come un framework di generazione full-stack possa essere integrato con pratiche orientate alla qualità del software.

6.2 SISTEMA DI TESTING ADOTTATO

Il caso di studio ha portato alla costruzione di un sistema di testing multilivello che combina strumenti e strategie di verifica differenti. L'obiettivo non è stato soltanto aumentare la copertura del codice, ma integrare diversi tipi di test in modo complementare, in modo da intercettare problemi a differenti livelli dell'applicazione.

Tabella 1: Livelli di test adottati nel progetto

Livello	Strumento	Obiettivo
Unit test frontend	Jest	Verifica dei componenti Angular
Integration test backend	Spring Boot Test	Verifica API REST e persistenza
Behavior test	Cucumber	Scenari funzionali ad alto livello
End-to-end	Cypress	Verifica comportamento dell'interfaccia
Mutation testing	PIT	Valutazione robustezza dei test
Static analysis	SonarQube	Analisi qualità del codice
Load testing	Gatling	Valutazione prestazioni

Questo approccio stratificato consente di individuare problemi a diversi gradi di granularità. I test più rapidi intercettano regressioni locali e problemi di implementazione, mentre i test end-to-end verificano il comportamento complessivo dell'applicazione dal punto di vista dell'utente.

6.3 CARATTERIZZAZIONE QUANTITATIVA DEL PROGETTO

Per contestualizzare i risultati ottenuti, la Tabella 2 riporta alcune metriche quantitative del progetto sviluppato durante il caso di studio.

Tabella 2: Caratteristiche quantitative del progetto

Metriche	Valore
Numero entità JPA	5
Numero repository	7
Numero service class	8
Numero REST controller	8
Componenti Angular	59
Righe di codice backend	6536
Righe di codice frontend	20571
Numero totale di commit	66
Durata sviluppo caso di studio	5 mesi

6.4 SUPPORTO AL TEST-DRIVEN DEVELOPMENT

Dal punto di vista del Test-Driven Development, JHipster offre un contesto favorevole soprattutto in termini di testabilità strutturale. L'architettura generata separa chiaramente dominio, persistenza, service layer, controller REST, e frontend, facilitando l'introduzione di test a diversi livelli.

Il framework genera automaticamente una suite iniziale di test che copre principalmente le operazioni CRUD, la serializzazione JSON e alcuni controlli di validazione. Questi test non derivano da un processo TDD in senso stretto, ma costituiscono una prima rete di sicurezza che consente di modificare l'applicazione con un rischio ridotto di regressioni.

Nel caso di studio, il TDD è stato applicato soprattutto durante l'evoluzione dell'applicazione, quando sono state introdotte funzionalità non previste dallo scaffolding iniziale. In questi casi il flusso di sviluppo ha seguito un approccio iterativo: definizione di un test di integrazione o di uno scenario comportamentale e successiva implementazione della logica necessaria per soddisfarlo.

Un aspetto osservato riguarda la copertura dei test generati automaticamente. Gran parte del backend viene verificata tramite test di integrazione, mentre i test unitari puri risultano meno diffusi. Questo approccio privilegia la verifica del comportamento complessivo delle API piuttosto che il testing isolato delle singole classi.

Un ulteriore limite osservato riguarda la copertura effettiva di alcune classi dell'applicazione. In particolare, diverse componenti generate automaticamente dal framework o appartenenti ai livelli infrastrutturali non sono coperte da test unitari specifici. Di conseguenza, tali classi risultano escluse dall'analisi di mutation testing effettuata con PIT, poiché i mutanti introdotti in queste porzioni di codice non vengono

esercitati dalla suite di test. Questo fenomeno evidenzia come la presenza di test generati automaticamente non garantisca una copertura completa dell'intero codice applicativo e richieda l'introduzione di test mirati qualora si voglia aumentare la robustezza complessiva della suite.

Nel complesso, JHipster non può essere considerato un framework *TDD-first*, ma fornisce una base fortemente predisposta alla testabilità e alla scrittura di test successivi allo scaffolding iniziale.

6.5 SUPPORTO AL BEHAVIOR-DRIVEN DEVELOPMENT E AI TEST E2E

Il supporto al Behavior-Driven Development risulta meno diretto. JHipster non genera automaticamente una suite significativa di scenari comportamentali, ma l'integrazione con Spring Boot consente di utilizzare strumenti come Cucumber senza particolari difficoltà.

Nel caso di studio, gli scenari BDD sono stati utilizzati principalmente per descrivere comportamenti funzionali dell'applicazione e verificare l'interazione tra diversi componenti del sistema.

il BDD ha avuto soprattutto un ruolo complementare. Da un lato, ha mostrato la fattibilità dell'integrazione; dall'altro, ha evidenziato che il beneficio di tale approccio dipende fortemente dal contesto organizzativo. In un progetto sviluppato individualmente, infatti, il valore aggiunto del BDD tende a ridursi.

Un ruolo particolarmente importante è stato svolto dai test end-to-end realizzati con Cypress. A differenza dei test unitari frontend, che verificano il comportamento interno dei componenti Angular spesso attraverso meccanismi di mocking, i test E2E permettono di osservare direttamente il comportamento dell'interfaccia utente.

Nel corso dello sviluppo, questi test si sono rivelati particolarmente utili nel rilevare problemi di integrazione tra frontend e backend e difetti di interfaccia che non emergono facilmente nei test unitari.

6.6 SUPPORTO ALLA CONTINUOUS INTEGRATION

L'ambito in cui JHipster mostra il supporto più solido è probabilmente quello della Continuous Integration. Il framework genera infatti un progetto già compatibile con strumenti moderni di build, testing e containerizzazione.

Nel caso di studio è stata realizzata una pipeline Jenkins locale articolata in più fasi di verifica:

- build dell'applicazione
- esecuzione dei test unitari e di integrazione

- generazione dei report JUnit, JaCoCo e cucumber
- analisi statica con SonarQube
- esecuzione opzionale di mutation testing e load testing
- costruzione dell'immagine Docker e pubblicazione in un registry locale

Questa pipeline è stata realizzata sfruttando sia gli script generati da JHipster sia strumenti DevOps standard, dimostrando come il framework sia progettato per integrarsi facilmente con infrastrutture di automazione.

6.7 EVOLUZIONE DELLE METRICHE DURANTE LO SVILUPPO

Per valutare l'impatto delle attività di testing e integrazione continua durante lo sviluppo, sono state confrontate alcune metriche chiave in tre momenti differenti del progetto:

- progetto generato automaticamente da JHipster
- progetto dopo la generazione delle entità di dominio
- versione finale sviluppata nel caso di studio

Tabella 3: Evoluzione delle metriche di qualità durante lo sviluppo

Metriche	Scaffold JHipster	Dopo entità	Versione finale
Backend coverage (JaCoCo)	81%	78%	72%
Frontend coverage (Jest)	87.67%	91.43%	71.27%
Mutation score PIT	16%	23%	20%
Numero test backend	43	71	72
Numero test frontend	59	87	81
Numero test E2E	8	12	12
Code issues SonarQube	5	5	19

6.8 EVOLUZIONE DEL FRAMEWORK DURANTE LO STUDIO

Durante la redazione di questa tesi è stata rilasciata una nuova major release del framework, JHipster 9.0.0, pubblicata nel marzo 2026. Questa versione introduce aggiornamenti significativi dello stack tecnologico, tra cui il supporto a Spring Boot 4, nuove versioni dei framework frontend e requisiti aggiornati per Java 21 e Node.js 22. La frequenza degli aggiornamenti evidenzia un ecosistema attivo e in continua evoluzione. Allo stesso tempo, l'introduzione di cambiamenti incompatibili tra versioni rappresenta una possibile sfida per la manutenzione di progetti esistenti. Per mitigare questo problema, JHipster mette a disposizione strumenti dedicati per l'aggiornamento automatico delle applicazioni, come il sub-generator `jhipster upgrade`, che consente di migrare gradualmente il codice generato verso versioni più recenti del framework. Nel contesto del presente caso di studio, l'applicazione è stata sviluppata utilizzando JHipster 8.11.0. L'aggiornamento alla versione 9 richiederebbe alcune modifiche all'ambiente di sviluppo e allo stack tecnologico, ma la presenza di strumenti di migrazione suggerisce che tali aggiornamenti possano essere gestiti con uno sforzo relativamente contenuto.

6.9 OSSERVAZIONI CRITICHE EMERSE DAL CASO DI STUDIO

L'esperienza di sviluppo ha evidenziato diversi punti di forza ma anche alcune difficoltà.

Un primo elemento positivo riguarda la rapidità con cui è possibile avviare un progetto completo. JHipster fornisce una base applicativa già configurata, nella quale backend, frontend e strumenti di build risultano immediatamente integrati. Questo consente di concentrarsi rapidamente sulla logica applicativa senza dover configurare manualmente l'infrastruttura del progetto.

Allo stesso tempo, questa ricchezza di componenti introduce una curva di apprendimento iniziale significativa. Per poter modificare il comportamento dell'applicazione è necessario comprendere il funzionamento di diversi framework e strumenti, tra cui Angular, Spring Boot, Maven, Docker e il sistema di build del frontend.

Durante lo sviluppo è emersa anche una differenza di trasparenza tra i framework utilizzati per frontend e backend. Nel caso di Angular, la separazione tra componenti, servizi e modelli applicativi è risultata relativamente intuitiva e facilmente modificabile. Al contrario, il backend basato su Spring Boot è apparso più opaco, poiché una parte significativa del comportamento dell'applicazione dipende da meccanismi di configurazione automatica e dependency injection. Questi meccanismi semplificano notevolmente lo sviluppo delle funzionalità comuni, ma possono rendere meno immediata la comprensione dei dettagli interni del framework e la diagnosi di problemi

meno evidenti.

Un ulteriore elemento di complessità riguarda la struttura del backend generato da JHipster. L'uso combinato di repository, service, DTO e mapper introduce un livello di astrazione utile per separare le responsabilità, ma può risultare inizialmente difficile da comprendere, soprattutto quando sono presenti relazioni tra entità e meccanismi di persistenza automatica.

Dal punto di vista del testing frontend, i test unitari basati su Jest utilizzano frequentemente meccanismi di mocking per verificare il comportamento interno dei componenti Angular. Sebbene questo approccio sia utile per test isolati, nel caso di studio i test end-to-end con Cypress si sono rivelati più efficaci per individuare problemi reali dell'interfaccia e dell'integrazione con il backend.

Per quanto riguarda la Continuous Integration, JHipster offre una base molto solida grazie alla presenza di configurazioni Docker e script di build già pronti per l'esecuzione in ambienti CI. Tuttavia, la realizzazione di una pipeline articolata richiede comunque una buona conoscenza degli strumenti DevOps coinvolti.

Nel complesso, il caso di studio mostra che JHipster rappresenta una piattaforma efficace per sviluppare applicazioni web complete e integrare pratiche orientate alla qualità del software. Il framework riduce notevolmente il tempo necessario per avviare un progetto e fornisce un ecosistema tecnologico coerente, ma richiede una fase iniziale di apprendimento per comprendere pienamente il funzionamento dei framework sottostanti.

CONCLUSIONI E SVILUPPI FUTURI

La presente tesi ha analizzato il ruolo dei framework di generazione full-stack nello sviluppo di applicazioni web moderne, con particolare attenzione al supporto che tali strumenti offrono alle pratiche di qualità del software. In particolare, il lavoro si è concentrato sul framework JHipster e sulla sua capacità di supportare metodologie come Test-Driven Development (TDD), Behavior-Driven Development (BDD) e Continuous Integration (CI) [2, 19, 8, 15].

L'indagine è stata condotta attraverso due prospettive complementari: da un lato l'analisi teorica delle metodologie e degli strumenti di testing e automazione; dall'altro la realizzazione e la valutazione empirica di un caso di studio concreto sviluppato a partire da uno scaffolding generato automaticamente da JHipster.

7.1 SINTESI DEL LAVORO SVOLTO

Nel corso della tesi è stata progettata e sviluppata un'applicazione web di gestione delle note basata su JHipster. Il progetto ha permesso di analizzare in modo pratico l'interazione tra codice generato automaticamente, personalizzazione del dominio applicativo e introduzione progressiva di pratiche di quality assurance.

L'applicazione sviluppata include funzionalità tipiche di un sistema di gestione delle note, tra cui creazione e modifica delle note, associazione di tag e gestione di allegati. Durante lo sviluppo sono state introdotte modifiche allo scaffolding iniziale per adattare il comportamento dell'applicazione alle esigenze del dominio e per sperimentare l'integrazione di diverse strategie di testing.

Il progetto ha integrato diversi livelli di verifica automatizzata, tra cui test di integrazione backend con Spring Boot Test, test frontend con Jest, scenari comportamentali con Cucumber, test end-to-end con Cypress, mutation testing con PIT e analisi statica del codice con SonarQube. Tali strumenti sono stati inseriti all'interno di una pipeline di Continuous Integration realizzata tramite Jenkins e Docker [4, 5, 20, 23, 14, 6].

Questo approccio ha consentito di osservare in modo concreto come un progetto generato automaticamente possa evolvere in un sistema dotato di un processo

di verifica strutturato e multilivello, in linea con le pratiche contemporanee di automazione della qualità del software [3].

7.2 RISPOSTA ALLA DOMANDA DI RICERCA

Alla luce dell'analisi teorica e sperimentale, si può concludere che JHipster supporta in modo concreto pratiche di sviluppo orientate alla qualità, anche se con livelli di supporto differenti a seconda della metodologia considerata.

Per quanto riguarda il Test-Driven Development, la struttura architetturale generata dal framework — basata su una separazione chiara tra livelli applicativi — favorisce la testabilità del codice e rende relativamente semplice introdurre test durante l'evoluzione dell'applicazione.

Nel caso del Behavior-Driven Development, il supporto offerto dal framework è più indiretto, e non genera automaticamente scenari comportamentali.

L'ambito in cui JHipster dimostra il supporto più solido è quello della Continuous Integration. Il framework fornisce infatti configurazioni già predisposte per build automatizzate, containerizzazione e integrazione con strumenti di analisi della qualità del codice. Questo rende relativamente semplice costruire pipeline di integrazione continua anche in ambienti di sviluppo locali o sperimentali.

In termini complessivi, la risposta alla domanda di ricerca può essere formulata come segue: JHipster non impone una metodologia specifica, ma costituisce un abilitatore tecnico efficace per l'adozione disciplinata di pratiche orientate alla qualità, con un supporto particolarmente forte alla Continuous Integration, buono alla testabilità e più indiretto al BDD e E2E.

7.3 PRINCIPALI CONTRIBUTI DELLA TESI

Il primo contributo di questo lavoro consiste nell'aver mostrato, attraverso un caso di studio concreto, come un framework di generazione automatica del codice possa essere integrato con metodologie di testing e automazione tipiche dello sviluppo software moderno.

Il secondo contributo riguarda la valutazione critica dell'interazione tra scaffolding automatico e pratiche test-driven. Il caso di studio ha evidenziato che il codice generato costituisce una base solida per avviare rapidamente un progetto, ma non sostituisce la necessità di progettare esplicitamente i test e le regole di business dell'applicazione.

Un ulteriore risultato emerso riguarda il ruolo dei diversi livelli di testing. Nel caso di studio i test end-to-end realizzati con Cypress si sono rivelati particolarmente efficaci nel rilevare problemi di integrazione tra frontend e backend e difetti dell'interfaccia utente che difficilmente emergono nei test unitari.

Infine, la tesi mostra che anche in un contesto locale, riproducibile e a costo contenuto è possibile costruire un processo di quality assurance articolato, integrando livelli diversi di test, analisi statica e automazione CI in modo coerente con un progetto generato da JHipster.

7.4 SVILUPPI FUTURI

Il lavoro svolto apre diverse possibili direzioni di approfondimento.

Un primo sviluppo naturale consiste nell'estendere l'analisi a configurazioni architetturali differenti di JHipster, in particolare al paradigma a microservizi. Questo permetterebbe di osservare come le pratiche di testing e Continuous Integration si comportino in presenza di maggiore complessità distribuita e comunicazione tra servizi.

Un secondo possibile sviluppo riguarda il rafforzamento della valutazione empirica. Sarebbe interessante confrontare il processo di sviluppo basato su JHipster con quello di applicazioni realizzate manualmente o con altri generatori full-stack, al fine di valutare con maggiore precisione l'impatto dello scaffolding automatico sulla qualità del software.

Dal punto di vista applicativo, il caso di studio potrebbe essere esteso introducendo funzionalità più complesse, come meccanismi di collaborazione tra utenti, sistemi di notifica o gestione avanzata dei permessi. Tali estensioni renderebbero il dominio applicativo più ricco e permetterebbero di analizzare scenari di testing ancora più articolati.

Infine, sul piano infrastrutturale, potrebbe risultare interessante portare la pipeline in un contesto cloud o semi-distribuito, integrando registry remoti, ambienti di staging, deployment automatizzato e monitoraggio post-rilascio. Infine, potrebbe risultare interessante portare la pipeline di integrazione continua in un ambiente cloud o distribuito, integrando deployment automatizzato, ambienti di staging e strumenti di monitoraggio dell'applicazione in produzione.

In conclusione, il caso di studio conferma che i framework di generazione full-stack, se utilizzati con consapevolezza metodologica, non rappresentano un'alternativa alle buone pratiche di ingegneria del software, ma possono costituire un efficace acceleratore per la loro adozione.

BIBLIOGRAFIA

- [1] Kent Beck. *Extreme Programming Explained: Embrace Change*. Addison-Wesley Professional, Boston, MA, 2001.
- [2] Kent Beck. *Test-Driven Development: By Example*. Addison-Wesley Professional, Boston, MA, 2002.
- [3] Lorenzo Bettini. *Test-Driven Development, Build Automation and Continuous Integration*. Leanpub, 2022. Accessed: 2026-03-13.
- [4] Cucumber. Cucumber documentation, 2024. Accessed: 2026-03-13.
- [5] Cypress.io. Cypress documentation, 2024. Accessed: 2026-03-13.
- [6] Docker Inc. Docker documentation, 2024. Accessed: 2026-03-13.
- [7] Viktor Farcic and Alex Garcia. *Test-Driven Java Development*. Packt Publishing, 2015.
- [8] Martin Fowler. Continuous integration, 2006.
- [9] GatlingCorp. Gatling documentation, 2026. Accessed: 2026-03-13.
- [10] Google. Angular documentation, 2026. Accessed: 2026-03-13.
- [11] Google Container Tools. Jib documentation, 2026. Accessed: 2026-03-13.
- [12] Hibernate Project. Hibernate orm user guide, 2026. Accessed: 2026-03-13.
- [13] Jez Humble and David Farley. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Professional, Boston, MA, 2010.
- [14] Jenkins Project. Jenkins user documentation, 2024. Accessed: 2026-03-13.
- [15] JHipster Team. Jhipster official documentation, 2024. Accessed: 2026-03-13.
- [16] Michael Jones, John Bradley, and Nat Sakimura. Json web token (jwt). Technical Report RFC 7519, Internet Engineering Task Force, 2015.
- [17] Robert C. Martin. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, Upper Saddle River, NJ, 2008.

BIBLIOGRAFIA

- [18] Gerard Meszaros. *xUnit Test Patterns: Refactoring Test Code*. Addison-Wesley Professional, Boston, MA, 2007.
- [19] Dan North. *Introducing bdd*, 2006.
- [20] PIT Mutation Testing. *Pit mutation testing documentation*, 2024. Accessed: 2026-03-13.
- [21] John Ferguson Smart. *BDD in Action: Behavior-Driven Development for the Whole Software Lifecycle*. Manning, Shelter Island, NY, 2014.
- [22] Ian Sommerville. *Software Engineering*. Pearson, Boston, MA, 10th edition, 2016.
- [23] SonarSource. *Sonarqube documentation*, 2024. Accessed: 2026-03-13.
- [24] VMware. *Spring boot reference documentation*, 2026. Accessed: 2026-03-13.
- [25] VMware. *Spring security reference documentation*, 2026. Accessed: 2026-03-13.